Java 虚拟机面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：[学编程、做项目、拿 Offer！]*
> 2. *企业高频面试题库：[开始刷题，面试遇原题！]*
> 3. *精选简历模板大全：[1 分钟搞定简历！]*
> 4. *AI 资源导航网站：[获取最新 AI 黑科技！]*
> 5. *1 对 1 模拟面试：[随时随地提升面试能力]*

JVM 方法区是否会出现内存溢出？

方法区在 JVM 里主要存类的元数据、常量池、字段方法信息这些。以前永久代（PermGen）时代，这地方确实会 OOM，尤其是动态生成类的场景，比如早期用 CGLIB 做字节码增强时，加载太多类就容易把永久代打满。

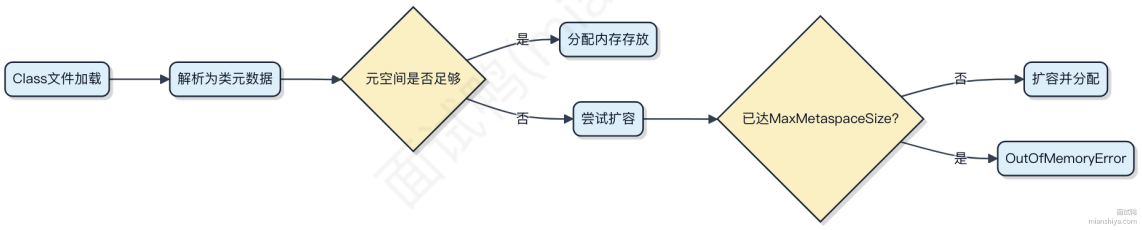
JDK 8 开始，**元空间**（Metaspace）替代了永久代，这块内存挪到了堆外，直接用的是系统内存。默认情况下，元空间可以动态扩容，上限是系统可用内存。但如果你显式通过 `-XX:MaxMetaspaceSize` 设了个上限，那一旦加载的类太多超过这个值，照样会 OutOfMemoryError。

所以答案是：会溢出。尤其在以下场景： 1）大量动态生成类，比如某些框架频繁做 ASM 字节码操作 2）部署很多应用的服务器上，每个应用都有一堆类，比如上百个微服务模块热部署 3）存在类加载器泄漏，老的 ClassLoader 卸不掉，对应的类元数据也回收不了

代码上没啥特别示例，但调优时注意这两个参数：

```
-XX:MaxMetaspaceSize=256m
-XX:MetaspaceSize=128m
```

监控的时候关注 `CommittedMBC` 和 `UsedMBC` 这两个指标，突然涨得快就得查是不是有类加载风暴。



你常用哪些工具来分析 JVM 性能？

分析 JVM 性能，关键得看场景。如果是线上突发问题，**Arthas** 是首选，不用重启应用就能直接连上去看堆栈、内存、线程，特别适合定位死锁或 CPU 突增。

本地调试或者压测时，JDK 自带的工具更轻量。 `jstat` 看 GC 频率和内存变化最直接，比如每秒 Young GC 次数超过 5 次就得警惕了。 `jstack` 抓线程堆栈，配合 `top -H` 找出 CPU 占高的线程，能快速定位到是哪个方法卡住了。 `jmap` 可以导出堆 dump，再用 **MAT** 或 JVisualVM 分析对象占用，查内存泄漏很管用。

如果要长期监控，一般会上 **Prometheus + Grafana**，配合 Micrometer 或 JMX Exporter 采集 JVM 指标。GC 次数、堆内存、线程数这些数据一目了然，还能设告警。

复杂问题还得靠飞行记录器。JFR（Java Flight Recorder）和 JMC 搭配，能拿到方法级的性能采样数据，像火焰图一样看出热点方法。不过这功能在 Oracle JDK 免费，OpenJDK 要从 11+ 开启额外参数。

简单问题别搞太重，`jstat` 和 `jstack` 组合拳基本够用。真要深挖，Arthas 和 JFR 才是杀手锏。

什么条件会触发 Java 的 young GC？

年轻代空间不足时就会触发 young GC，这是最直接的条件。JVM 的堆被分成年轻代和老年代，对象优先在年轻代的 Eden 区分配，当 Eden 区快满的时候，系统会进行一次 minor GC，也就是 young GC。

1）Eden 区满了是主要触发时机。比如默认情况下 Eden 和两个 Survivor 区的比例是 8:1:1，一旦 Eden 被填满，不管老年代还有多少空间，都会触发一次回收。

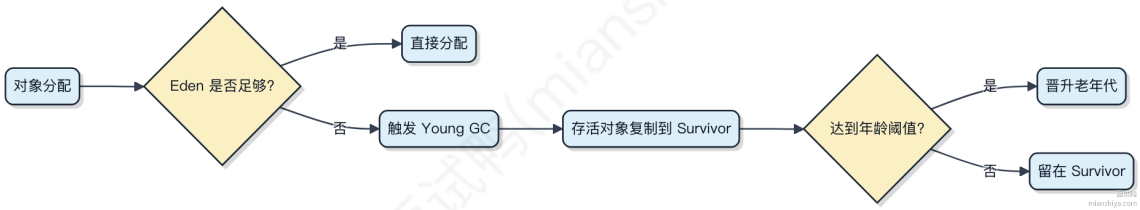
2）大对象直接进入老年代的情况也可能间接影响，但不会直接触发 young GC。如果频繁分配大对象导致老年代压力大，可能会加快 Full GC 的到来，但这不是 young GC 的触发条件。

3）代码里其实没法主动“控制”什么时候发生，只能通过对象分配行为影响。比如连续创建大量临时对象：

```
for (int i = 0; i < 100000; i++) {
    byte[] data = new byte[1024]; // 每次分配 1KB 对象
}
```

这种写法很快就能把 Eden 塞满，从而观察到 young GC 的日志。

young GC 的核心目标是清理掉那些朝生夕死的对象，把还活着的对象挪到 Survivor 区或者晋升到老年代。整个过程采用的是**复制算法**，效率比较高，暂停时间短，所以设计上允许它频繁发生。



JVM 的 TLAB（Thread-Local Allocation Buffer）是什么？

每个线程在堆上分配对象时，如果每次都去竞争共享的 Eden 区，那效率会很低。JVM 为了减少锁争用，给每个线程单独划了一小块私有内存区域，这就是 **TLAB**。

它本质是 Eden 区里的一段缓存，线程可以快速在自己的 TLAB 中分配对象，不用加锁。只有当 TLAB 空间不足时，才会重新申请新的 TLAB，这时候才需要同步。

1）TLAB 默认是开启的，由 `-XX:+UseTLAB` 控制
2）大小不是固定的，通过 `-XX:TLABSize` 可设置初始大小，JVM 也会动态调整

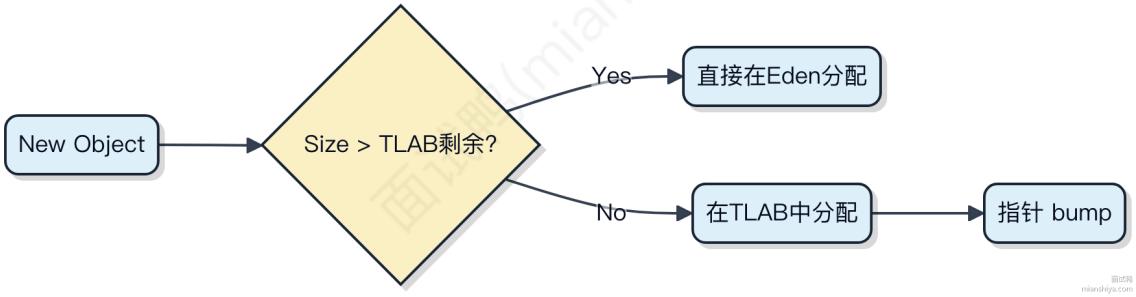
3）如果对象太大，直接在共享区分配，避免浪费 TLAB 剩余空间

代码层面你看不到直接操作 TLAB 的 API，它是 JVM 自动管理的。但你可以通过 GC 日志观察它的使用情况：

```
-XX:+PrintGCDetails -XX:+PrintTLAB
```

日志里会输出每个线程的 TLAB 分配速率、浪费空间等信息，用来调优。

有个关键点：TLAB 回收不是垃圾回收做的，而是随着线程结束或 TLAB 被填满后，整个归还给 Eden 区统一管理。所以它不解决对象存活问题，只解决**分配时的并发冲突**这个脏活累活。



TLAB 在高并发创建对象的场景下特别有用，比如 Dubbo 这种 RPC 框架处理大量请求时，临时对象多，有 TLAB 能明显降低分配开销。

什么是 Java 的 PLAB？

PLAB 是 JVM 在多线程环境下为对象分配内存时用到的本地缓冲区，主要发生在新生代的 Eden 区。

多线程同时触发 Minor GC 时，每个线程都需要在 Eden 区分配对象空间。如果直接操作共享的 Eden 区，会因为竞争锁带来大量开销。JVM 为每个线程分配一块私有的小内存区域，也就是 **PLAB**（Thread Local Allocation Buffer），让线程先在自己的缓冲区里分配对象，这样就不需要频繁加锁了。

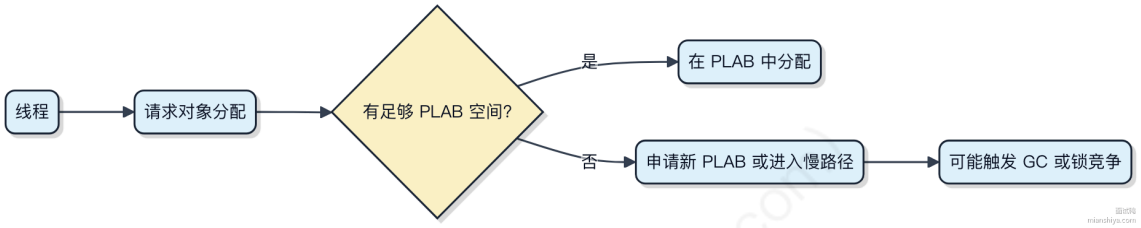
每个 PLAB 大小不固定，一般在几 KB 到几十 KB 之间，由 JVM 自动调整。当线程的 PLAB 用完后，会申请新的 PLAB，旧的被回收或归还。如果对象太大，直接在共享区域分配，或者触发 TLAB 填充失败处理逻辑。

这种机制能显著减少多线程分配时的竞争，提升吞吐量。但也会带来一点内存浪费，比如 PLAB 剩余空间不够分配下一个对象，这块碎片就闲置了。JVM 会控制这个浪费比例，默认不超过 1%。

代码上我们无法直接操作 PLAB，但可以通过参数调优：

```
-XX:+UseTLAB        // 开启 TLAB（默认开启）
-XX:TLABSize=xx     // 设置初始大小
-XX:+PrintTLAB      // 打印 TLAB 使用情况，用于诊断
```

1）GC 线程也会有自己的 PLAB
2）PLAB 仅用于对象分配，不用于晋升或老年代分配
3）所有行为由 JVM 自动管理，应用层无感知



为什么 Java 中某些新生代和老年代的垃圾收集器不能组合使用？比如 ParNew 和 Parallel Old

不同垃圾收集器之间能不能组合使用，关键看它们的 **设计架构** 和 **并发模型** 是否兼容。JVM 的 GC 实现不是随便拼凑的，每个收集器都有一套自己的工作方式，尤其是跨代引用处理和停顿控制这块。

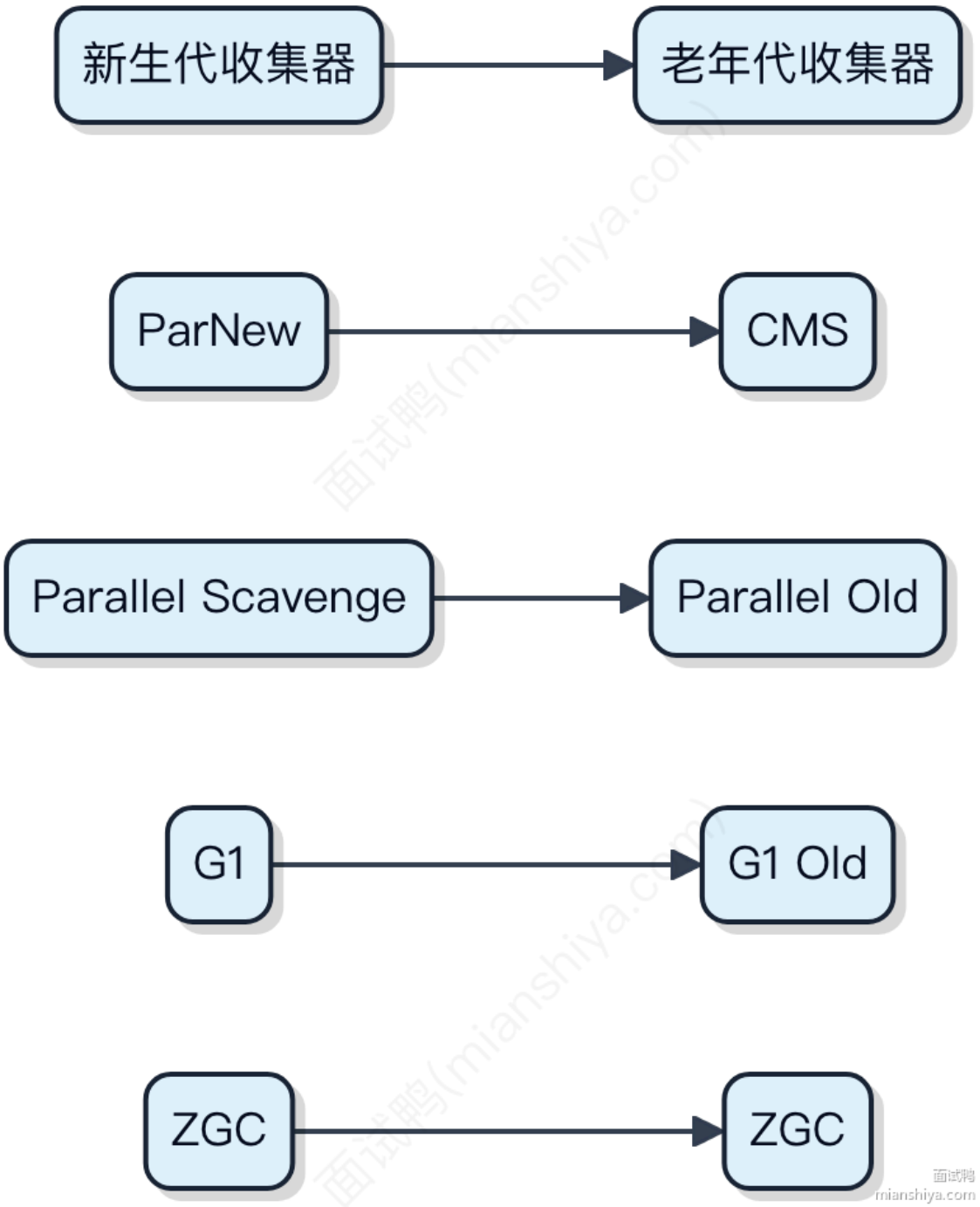
1）新生代和老年代的收集器要能配合，得共享一套 GC 根扫描机制和线程协作模型。比如 CMS 和 ParNew 能搭，是因为 ParNew 专门加了对 CMS 的支持代码，两者都考虑到了并发阶段如何安全地处理跨代指针。

2）Parallel Scavenge 和 Parallel Old 是一对，它们共用一套自动调优策略和并行控制逻辑，追求的是最大吞吐量，压根不打算支持 CMS 或 G1 这类并发型老年代收集器。

3）ParNew 虽然是并行的新生代收集器，但它只服务于 CMS 模式下的老年代回收。你没法用它配 Parallel Old，因为后者完全没实现跟 ParNew 协作的接口，连触发机制都不一样 —— Parallel Old 是 STW 全堆回收，而 CMS 是并发标记。

4）从 JDK 8 开始，官方推荐直接用 G1，它把整个堆统一管理，不再分这种 tight-coupled 的组合。像 ZGC、Shenandoah 更是彻底去掉分代概念的趋势。

面试鸭(mianshiya.com)



简单说，不是技术上搞不定，而是 Oracle 只实现了特定组合，其他没写的压根不经过测试，也不保证可用。

JVM 新生代垃圾回收如何避免全堆扫描？

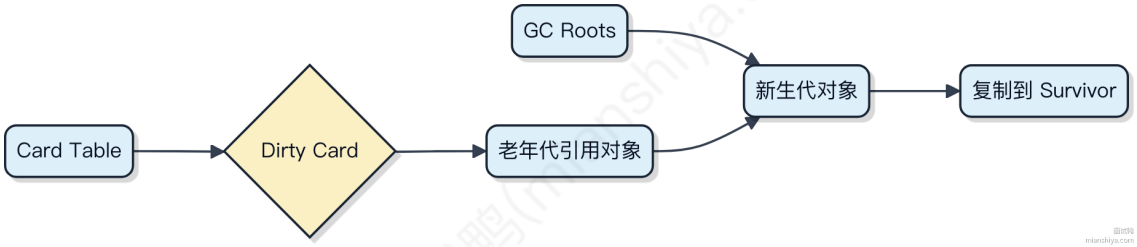
新生代回收能高效运行，关键在于它压根不碰老年代的对象。JVM 用了一种叫**分代收集**的设计，把堆分成新生代和老年代，大部分对象朝生夕死，集中在新生代。

1）新生代回收采用复制算法，只扫描新生代区域。它从 GC Roots 出发，但有个前提：如果老年代对象引用了新生代对象，这个引用关系必须被记录下来，否则就会漏掉存活对象。

2）Card Table（卡表）就是干这事的。它把老年代内存划成 512 字节一页，用一个字节标记页是否“脏”。只要老年代对象修改了对新生代的引用，对应卡片就被标为 dirty。

3）YGC 时，除了常规的 GC Roots，还会把 dirty 的卡片对应的区域也作为 Roots 扫描，确保跨代引用不被遗漏。这样既不用全堆扫描，又不会误删存活对象。

像 CMS、G1 都依赖这套机制。G1 虽然自己有 Remembered Set，但原理类似，只是粒度更细。



整个过程新生代单独完成，停顿时间控制在 10~50ms 级别，靠的就是这种隔离 + 增量更新的策略。

什么是 Java 中的 logging write barrier？

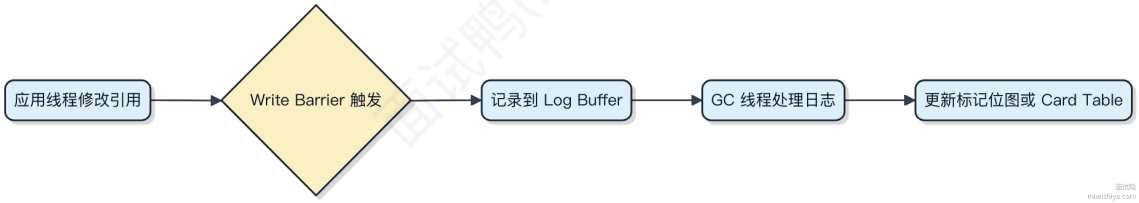
Java 中的 logging write barrier 其实是垃圾回收过程中用来追踪对象引用变化的一种机制，主要用在像 G1 或 ZGC 这类关注并发标记的收集器里。

它不直接“记录日志”到文件，而是把对象引用的修改，比如 `obj.field = newRef`，这种写操作记下来。因为并发标记阶段用户线程还在跑，对象图可能随时变，**write barrier** 就是在每次引用字段被修改时插一段逻辑，把这次变更记录到一个队列里。

这些记录后续会被并发标记线程消费，确保新引用的对象不会被漏标，从而保证标记的准确性。你可以把它看作是 GC 对“谁改了引用”这件事的监控探针。

1）写屏障代码通常由 JVM 自动生成，不需要开发者干预
2）它带来的开销很小，但高频调用场景下仍会有一定性能影响
3）G1 使用 post-write barrier 来记录 card table，ZGC 则用 load barrier 配合指针着色

举个简单例子，G1 里一旦发现某个 region 的引用被更新，就会通过 write barrier 标记对应 card，让后续 Remark 阶段能快速扫描脏卡。



这机制说白了就是为了解决“一边干活一边记账”的一致性问题，让 GC 能在不停顿的情况下安全完成标记。

JVM 垃圾回收调优的主要目标是什么？

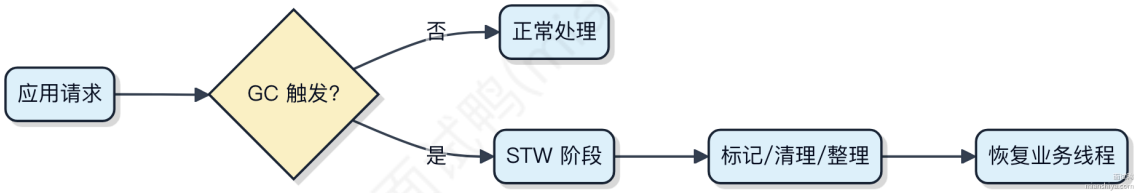
JVM 垃圾回收调优不是为了把 GC 时间压到零，那不现实。目标很实际：在应用的吞吐量、延迟和内存占用之间找到平衡点。

1） 吞吐量优先的场景，比如离线计算任务，像 Spark 批处理这种，可以选 G1 或 Parallel GC，目标是让用户代码跑得更久，GC 停顿少打断。

2） 低延迟是关键的应用，比如交易平台下单链路，每次 GC 停顿必须控制在 100ms 内，这时候 ZGC 或 Shenandoah 就更合适，它们能把 STW（Stop-The-World）拆成小段，压根不经过长时间卡顿。

3） 内存 footprint 小也很重要，特别是在容器化环境，K8s 里跑着几百个 Java 实例，用 ZGC 虽然停顿短，但内存开销大，可能反而不如 G1 稳定。

调的时候常见翻车点是堆设太大，结果老年代回收拖不动，一次 Full GC 几秒直接超时。一般来说，新生代别超 4G，避免 Minor GC 时间过长。



重点盯住 GC 日志里的 Pause Time 和 Frequency，配合监控看 P99 延迟有没有毛刺。调优到最后，其实是业务容忍度和硬件成本的权衡。

如何对 Java 的垃圾回收进行调优？

调优垃圾回收不是上来就调参数，而是先搞清楚应用的 GC 行为。大部分问题其实出在对象创建太猛，而不是 GC 本身不行。

1）先用 `jstat -gc` 或 `jcmd <pid> GC.run_finalization` 观察频率和耗时。如果 Young GC 超过 50ms 或频繁到几十毫秒一次，说明对象分配速率太高，得查代码里有没有短生命周期大对象。

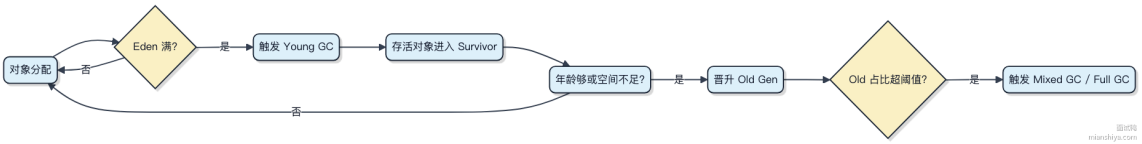
2）老年代增长快？可能是内存泄漏，也可能是晋升太快。打开 `-XX:+PrintGCDetails` 看对象从 Eden 到 Old 的晋升日志。可以加 `-XX:MaxTenuringThreshold=1` 强制提前晋升测试是否扛得住。

3）停顿时间敏感的应用，比如接口响应要求 200ms 内，CMS 已经过时了，直接上 **ZGC** 或 **Shenandoah**。它们的暂停时间基本稳定在 10ms 以内，哪怕堆到 32GB 也压不垮。

```
// 启动参数示例：用 ZGC，堆设 8G
-XX:+UseZGC -Xmx8g -XX:+UnlockExperimentalVMOptions
```

4）别迷信“最大吞吐量”。吞吐型服务可以用 G1，但记得设置 `-XX:MaxGCPauseMillis=200` 目标停顿，JVM 会自动调整分区回收数量。

真正要动手调的，一般就三个方向：控制对象分配速率、减少晋升压力、选对收集器。工具链推荐配合 Arthas 的 `vm.gc` 命令做线上诊断，比 jstat 更直观。



常用的 JVM 配置参数有哪些？

JVM 参数选对了，服务能少掉一半链路。先分三类来看，最常用的是内存相关。

堆内存一般用 -Xms 和 -Xmx 设成一样，避免动态扩容开销，比如 -Xms2g -Xmx2g。年轻代大小用 -Xmn，通常设为堆的 1/3 到 1/2。如果对象多，可以调大一点。

垃圾回收器得看场景。响应优先用 **G1**，加 -XX:+UseG1GC 就行，再配 -XX:MaxGCPauseMillis=200 控制停顿时间。吞吐优先就上 ParallelGC，-XX:+UseParallelGC，适合后台批处理。ZGC 现在用得多，-XX:+UseZGC 能把 STW 压到 10ms 内，但得 JDK 15+。

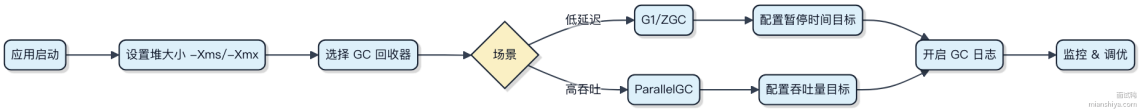
日志参数不能少。-Xlog:gc* 开启 GC 日志，最好加上 time,uptime,level,tags 打全信息。线上出问题靠这个定位。

还有几个关键开关。-XX:+HeapDumpOnOutOfMemoryError 自动导出堆 dump，配合 -XX:HeapDumpPath=/path 存文件。-XX:+PrintCommandLineFlags 启动时打印实际生效的参数，排查配置有没有被覆盖。

别乱调 -XX:MaxMetaspaceSize，除非你真遇到元空间爆了。多数时候让它自动扩就行。

代码示例：

```
java -Xms4g -Xmx4g -Xmn2g -XX:+UseG1GC -XX:MaxGCPauseMillis=200 \
     -Xlog:gc*:gc.log:time,uptime:filecount=5,filesize=100m \
     -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dump MyApp
```



编译执行与解释执行的区别是什么？JVM 使用哪种方式？

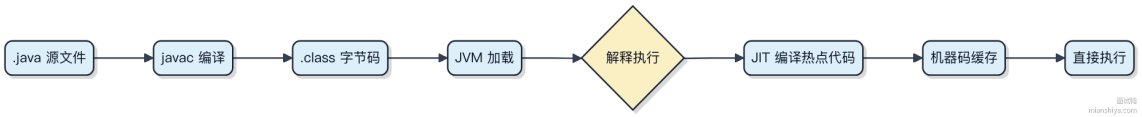
Java 代码的执行混合了编译和解释两种方式，得从整个流程来看。

1）源代码先由 `javac` 编译成字节码（.class 文件），这个阶段是**编译执行**的一部分。但这里的“编译”不是直接生成机器码，而是生成一种中间形态的字节码，平台无关。

2）运行时 JVM 读取字节码，再通过解释器逐行解释成机器指令执行。这就是**解释执行**，启动快，但执行慢，因为每次都要翻译。

3）为了提升性能，JVM 还引入了 **JIT（Just-In-Time）编译器**。它会在运行时把热点代码（频繁执行的方法）直接编译成机器码缓存起来，下次调用就不用解释了，直接执行本地代码。

所以 JVM 实际上是“解释 + 编译”的混合模式。早期 HotSpot VM 就是靠这种策略兼顾了启动速度和运行效率。



像 Spring Boot 应用刚启动时接口响应慢，过一会儿变快，就和 JIT 的预热过程有关。GraalVM 提供的 AOT（提前编译）可以进一步优化这个问题，把部分工作提前到构建阶段。

如何在 Java 中进行内存泄漏分析？

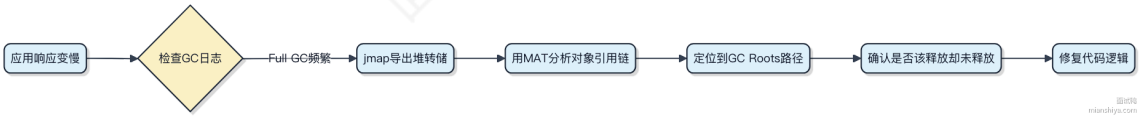
内存泄漏分析其实分两步走，先定位现象，再找根因。

应用跑着跑着 Full GC 频繁，老年代使用率蹭蹭往上涨，基本就能怀疑内存泄漏了。这时候得抓堆转储文件，用 jmap 命令就行，比如 `jmap -dump:format=b,file=heap.hprof <pid>` ，拿到 .hprof 文件后丢给分析工具。

主流工具就两个，**MAT** 和 JVisualVM。MAT 对泄漏感知更强，能自动给出 "Leak Suspects" 报告，直接告诉你哪些对象占得多、谁在强引用它们。比如你看到一堆 HashMap$Entry 挂着成千上万的业务对象，而 GC Roots 是某个静态缓存，那问题八成出在缓存没设上限或者没做清理策略。

常见泄漏场景也就那几类。静态集合类持有对象不放，比如 public static List 缓存数据但不清空；监听器和回调注册后不注销，Spring 里EventListener 注解用多了容易踩这坑；还有线程池创建线程时，ThreadLocal 没清理，堆栈里 Thread 数量持续增长，最后 OOM。

代码层面要防泄漏，核心是控制生命周期。集合类记得考虑 WeakHashMap 或 Guava Cache 这种带回收策略的；用 try-with-resources 确保流关闭；自定义线程池别瞎 new Thread，用 ThreadPoolExecutor 显式管理资源。



说到底，内存泄漏不是 JVM 的锅，是代码里那些“忘了释放”的引用在搞事。

JVM 由哪些部分组成？

JVM 主要由类加载子系统、运行时数据区、执行引擎和本地库接口组成。

1）类加载子系统负责把 .class 文件加载到内存。它包含加载、验证、准备、解析和初始化五个阶段。比如 Spring Boot 应用启动时，就是靠它把成千上万个类读进 JVM。

2）运行时数据区是核心内存布局，分为方法区、堆、虚拟机栈、本地方法栈和程序计数器。其中**堆**是最大的一块，存放对象实例和数组，GC 主要就在这里干活。方法区存常量、静态变量、类信息，JDK 8 后用元空间实现，直接扔到了本地内存里。

3）执行引擎负责执行字节码。解释执行逐条翻译，JIT 编译则把热点代码编译成机器码缓存起来，像 HotSpot 就靠这个提升性能。GC 线程也属于执行引擎的一部分，专门处理垃圾回收的脏活累活。

4）本地库接口（JNI）让 Java 调用 C/C++ 代码。比如你用 NIO 操作文件，底层其实是通过 JNI 调用了操作系统 API。

整个流程串起来：类加载器把字节码载入内存，执行引擎读取并运行，需要本地功能时通过 JNI 跳出去。堆和栈的划分决定了对象生命周期和线程隔离方式，这块搞不定，线上 OOM 问题基本扛不住。

JVM 有那几种情况会产生 OOM（内存溢出）？

JVM 产生 OOM 的情况主要集中在几个内存区域，每个区域的溢出都有特定场景和表现。

1）堆内存溢出是最常见的，对象持续创建但无法回收，最终超过 `-Xmx` 设置的上限。比如缓存没加过期策略，或者一次加载几百万条数据到 `List`，很容易把堆撑爆。

2）虚拟机栈溢出一般由线程请求深度超过限制引起，典型就是递归太深或死循环。每个线程默认栈大小是 1MB 左右（取决于系统），线程数太多也会耗尽内存，这种情况其实是**栈内存不足**而不是堆问题。

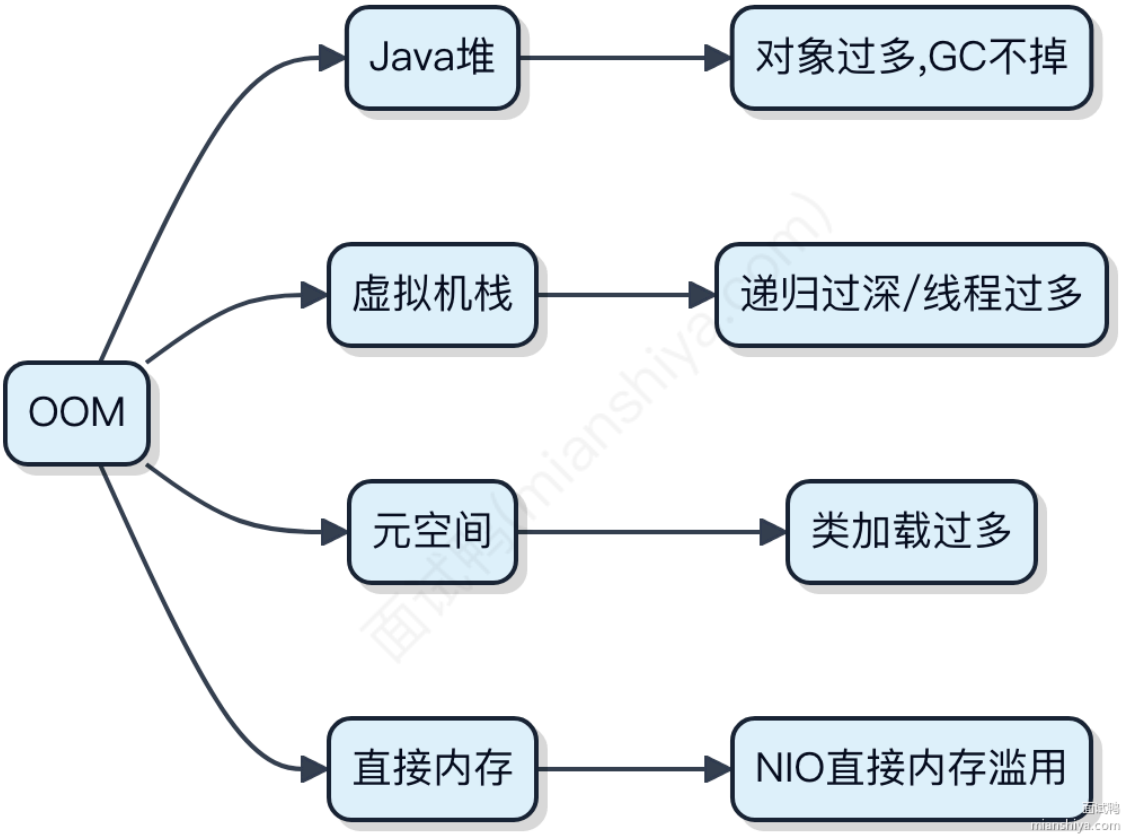
3）方法区（元空间）溢出在 Java 8 后主要是永久代被移除，改用本地内存存储类信息。如果动态生成大量类，比如 CGLIB 做字节码增强时没控制，元空间不断扩张又不卸载，就会 OOM。

4）直接内存溢出发生在使用 `ByteBuffer.allocateDirect()` 时，这部分内存不受 GC 控制，也不算在堆内，靠 `-XX:MaxDirectMemorySize` 限制。NIO 使用不当，频繁分配直接内存且迟迟不释放，就可能触发。

代码示例：

```
// 直接内存溢出模拟
for (;;) {
    ByteBuffer.allocateDirect(1024 * 1024); // 每次分配 1MB
}
```

这类问题线上排查得看 `OutOfMemoryError` 后面的提示，比如 "Java heap space"、"Metaspace"、"Direct buffer memory" 都能直接定位区域。



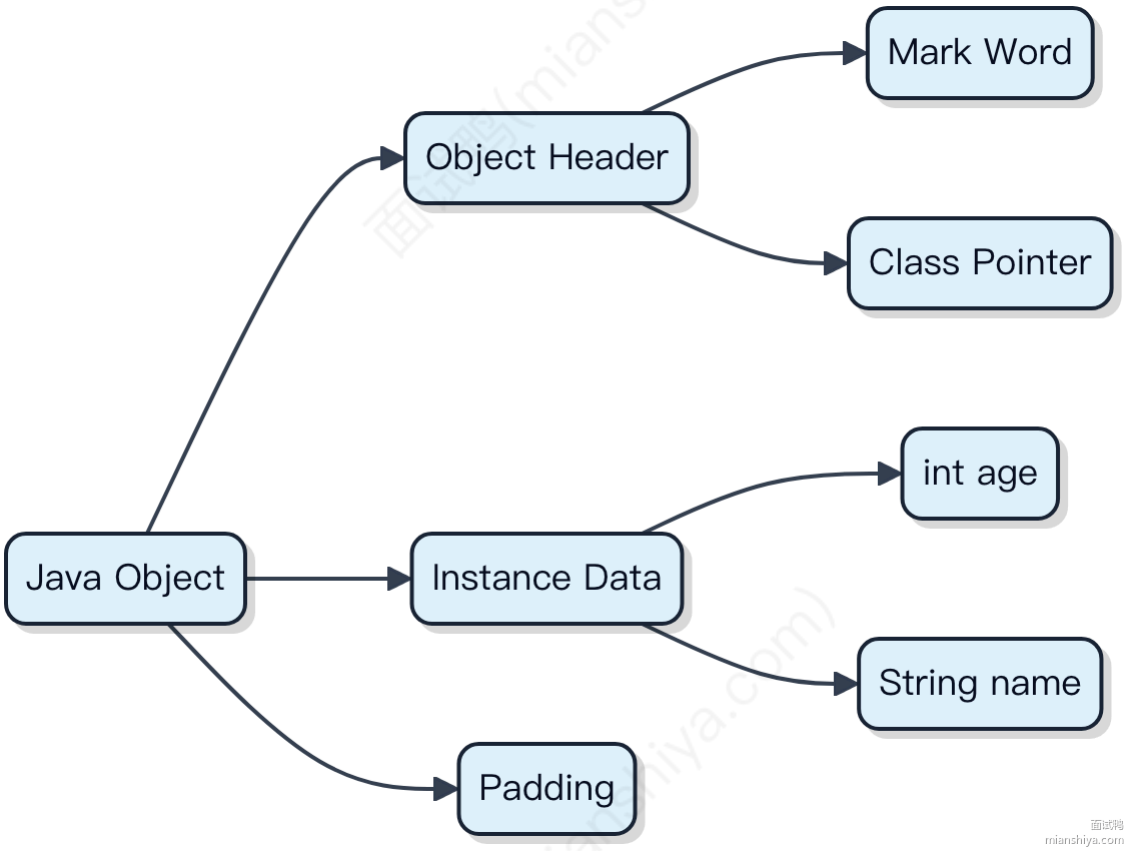
[Java 里的对象在虚拟机里面是怎么存储的？](#)

Java 对象在虚拟机里的存储，得从内存布局说起。对象创建后主要分布在堆上，而它的内存结构可以拆成三块：对象头、实例数据和对齐填充。

对象头里存了两样东西，一个是 Mark Word，用来放哈希码、GC 分代年龄、锁状态这些信息；另一个是指向类元数据的指针，JVM 靠它确定这个对象是哪个类的实例。如果是数组，这里还会多出一个记录长度的字段。

接下来是实例数据，这部分最实在，就是你定义的那些成员变量，按声明顺序排列，父类的变量也会包含进来。JVM 会按 long/double、int、short/char、byte/boolean 这样的顺序重新排序，目的是为了内存对齐，减少空间浪费。

最后是对齐填充，不是必须的。因为 JVM 要求对象大小必须是 8 字节的整数倍，不够就得补上。



这种布局方式直接影响 GC 和并发性能。比如 synchronized 锁升级过程就会修改 Mark Word 的内容，从无锁到偏向锁、轻量级锁再到重量级锁，都是在这块做文章。理解这块结构，对排查竞争激烈时的性能问题很有帮助。

JIT 编译后的代码存在哪？

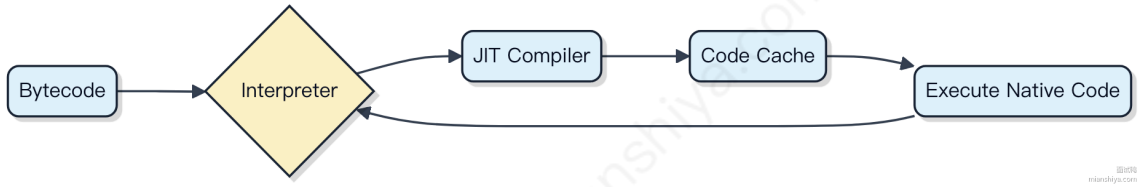
JIT 编译后的代码存在**代码缓存区（Code Cache）**，这是 JVM 堆外的一块内存区域，专门用来存放动态生成的本地机器码。

Java 程序启动时解释器逐条执行字节码，热点方法被 JIT 识别并编译成机器码。这些机器码不是放在堆里，也不是方法区，而是直接刷进 Code Cache。下次调用这个方法，虚拟机就直接跳过去执行这段本地代码，压根不经过解释器，性能提升明显。

不同 JVM 实现对 Code Cache 的管理策略不一样。HotSpot 默认大小几 MB 到上百 MB 不等，可以通过 `-XX:ReservedCodeCacheSize` 调整。一旦填满，JIT 编译就会停掉，后续方法只能解释执行，应用峰值性能会掉一截。

1）Client Compiler（C1）生成的代码相对简单，体积小，适合短生命周期的方法
2）Server Compiler（C2）或 Graal 编译出的优化程度高，代码更长，更容易撑满缓存

常见于高并发服务如 Dubbo 或 Spring Boot 应用跑久了出现 "CodeCache is full" 日志，这时候得看是不是编译策略不合理，或者缓存设太小。



说说 Java 的执行流程?

Java 程序跑起来其实就四步：写代码、编译、加载、执行。

你写的 .java 文件，先用 javac 编译成 .class 字节码。这一步不生成机器码，只生成一套能在任何平台跑的中间指令，保证了“一次编写，到处运行”。

字节码文件扔给 JVM 后，类加载器把它们按需加载进方法区。加载完，JVM 启动 main 线程，程序计数器和虚拟机栈跟着初始化。每个方法调用对应一个栈帧，压到线程的栈上。

真正执行靠**解释器**和**即时编译器（JIT）** 配合。解释器逐行读字节码解释执行，启动快但慢。JIT 发现某些方法被频繁调用（比如循环里），就把它编译成机器码缓存起来，下次直接执行本地代码，这就是热点代码优化。

GC 负责堆内存回收，线程私有的部分比如栈，随线程消亡自动释放。整个过程由 JVM 统一管控，开发者不用手动管内存布局和指令调度。

注意，JDK 8 之后元空间取代永久代，用的是本地内存，不再受 -XX:MaxPermSize 限制。

Java G1 相对于 CMS 有哪些进步的地方?

G1 出来就是为了解决 CMS 在大堆场景下停顿时间不可控的问题。CMS 虽然能实现大部分并发回收，但**停顿时间**没法保证，G1 的目标是让你能指定 -XX:MaxGCPauseMillis=200，它就尽量在 200ms 内搞定一次 GC。

G1 把整个堆拆成多个大小相等的 Region，不再有固定的年轻代和老年代内存块，代可以动态伸缩。回收的时候优先挑垃圾最多的 Region 先清，这就是“Garbage-First”名字的由来。这样能在有限时间内尽可能多回收空间，做到可预测的停顿。

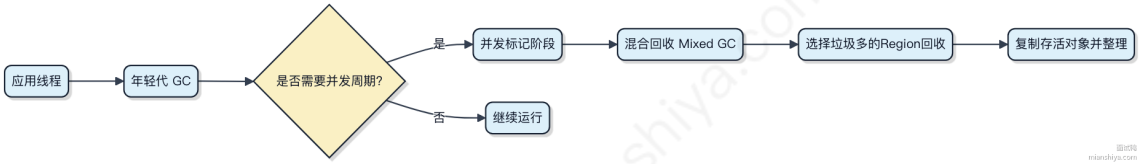
1）不再用 CMS 的 mark-sweep 算法，避免碎片问题。G1 在回收阶段会做部分整理，合并存活对象，减少后续 Full GC 风险。

2）Card Table 换成了 Remembered Set（RSet），每个 Region 自己维护谁引用了我。跨 Region 引用写屏障记录，避免扫描整个老年代。

3）并发标记阶段和 CMS 类似，但 G1 多了个“并发转移”阶段，真正回收时只选几个 Region 做 evacuation，复制到其他 Region，顺便整理内存。

4）CMS 一旦发生 Concurrent Mode Failure 就会退化成 Serial Old，整个应用卡死很久。G1 设计上更稳，配合 Humongous Region 处理大对象，降低 Full GC 概率。

代码上基本不用调，G1 自己根据停顿目标自适应调整回收策略。像 RocketMQ、Kafka 这些大堆服务现在基本都用 G1 或 ZGC。



简单说，G1 用 Region + RSet + 可预测停顿模型，把 GC 从“尽力而为”变成了“有承诺的服务”。

Java 是如何实现跨平台的？

Java 的跨平台能力，关键在于 **JVM** 和字节码的设计。你写的是 Java 代码，但跑在不同系统上的不是它，而是编译后的 `.class` 文件。

1）Java 源文件通过 `javac` 编译成字节码（.class），这玩意是中间指令，不依赖任何具体操作系统或 CPU 架构。

2）每个平台（Windows、Linux、macOS）都有对应的 JVM 实现。JVM 负责把字节码翻译成当前系统的机器指令，这个过程可以是解释执行，也可以通过 JIT 编译成本地代码来提速。

3）所以一次编写，到处运行——只要目标机器装了对应版本的 JVM，你的程序就能跑，底层差异全被 JVM 挡住了。

比如你在 Mac 上写的 Spring Boot 应用打包成 jar，在 Linux 服务器上照样能用 `java -jar` 启动，压根不经过重新编译。

```
// 编译后生成 Test.class
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello, JVM");
    }
}
```

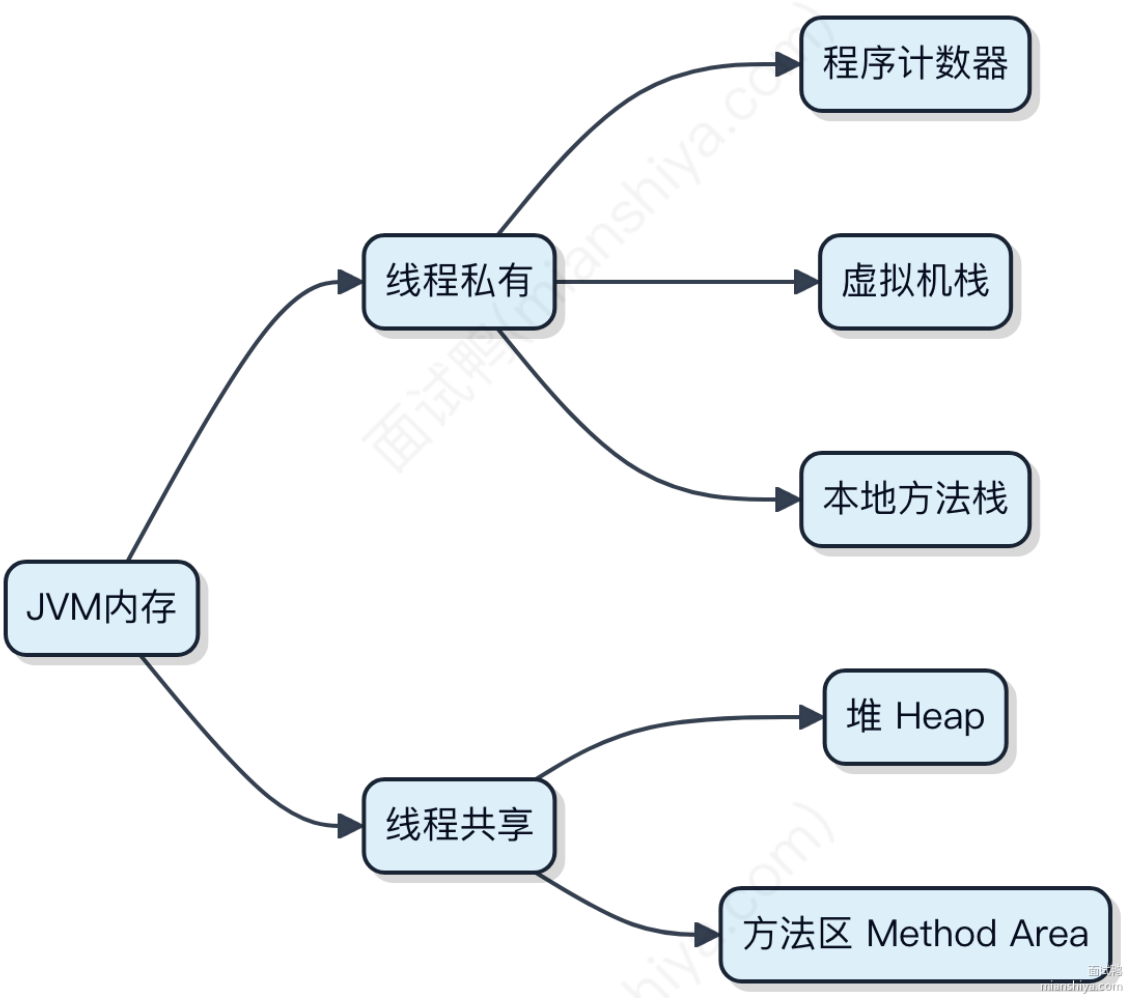
不同厂商的 JVM 实现比如 HotSpot（Oracle）、OpenJ9（IBM），它们对字节码的优化策略不同，但都遵循同一个规范，保证行为一致性。

JVM 的内存区域是如何划分的？

JVM 内存区域划分，其实是为了高效管理程序运行时的数据。不同数据生命周期不一样，比如方法执行的局部变量和整个类的结构信息，肯定不能混着放。

线程私有的部分包括 **程序计数器**、虚拟机栈和本地方法栈。程序计数器记录当前线程执行到哪条字节码指令了，线程切换回来能接着执行。每个方法调用时都会创建一个栈帧压入虚拟机栈，里面存了局部变量表、操作数栈、动态链接这些，方法执行完就弹出。本地方法栈则是为 native 方法服务的。

线程共享的区域主要是 **堆** 和 **方法区**。堆是最大的一块，几乎所有对象实例都在这里分配内存。GC 主要就是管这块区域，像 G1、ZGC 这些垃圾回收器优化的也是它。方法区存放类信息、常量、静态变量等，JDK 8 以后用元空间实现，放在本地内存里，避免永久代时代因加载类过多导致 OOM。



直接内存虽然不在 JVM 规范里定义，但 NIO 的 ByteBuffer 分配会用到，得小心 OutOfMemoryError，毕竟不受堆大小限制。

Java 中有哪些垃圾回收算法？

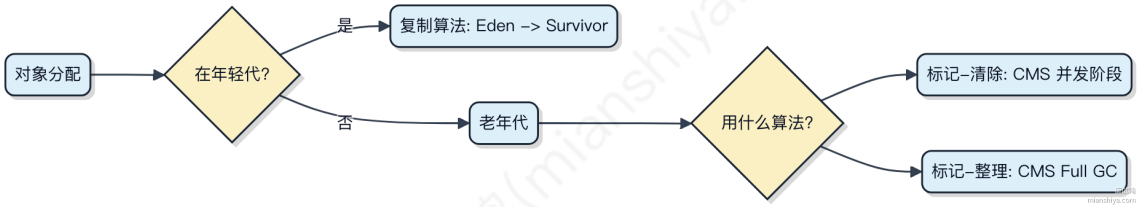
Java 的垃圾回收算法主要围绕怎么找垃圾、怎么清理垃圾展开。不同算法适合不同场景，一般 CMS、G1 这些收集器会组合使用多种算法。

1）标记-清除（Mark-Sweep）
先标记所有存活对象，然后统一回收未被标记的垃圾。最大的问题是会产生内存碎片，比如老年代用 Serial 收集器时就用这个算法，长时间运行后可能因为碎片触发 Full GC。

2）复制（Copying）
把内存分成两块，每次只用其中一块。当这一块满了，就把存活对象复制到另一块，然后清空原来的那块。效率高，不会产生碎片，但可用内存直接砍半。年轻代的 Minor GC 基本都用这个思路，比如 ParNew 和 G1 在年轻代阶段。

3）标记-整理（Mark-Compact）
标记过程和“标记-清除”一样，但后续不是直接清理，而是把存活对象往一端滑动，再清理边界外的内存。适合老年代这种对象多、回收少的场景，避免碎片问题。CMS 在并发失败降级后就会用这个算法做 Full GC。

4）分代收集（Generational Collection）
这不是具体算法，而是策略。基于对象生命周期分布：90% 对象朝生夕死。年轻代用复制算法，回收快；老年代用标记-清除或标记-整理。G1 和 ZGC 都是基于分代思想演进出来的。



像 G1 会把堆拆成多个 Region，逻辑上保留分代，回收时优先选垃圾多的区域，实现“增量整理”。ZGC 和 Shenandoah 更激进，主打并发整理，停顿时间控制在 10ms 以内。

Java 中堆和栈的区别是什么？

Java 里堆和栈是两种不同的内存区域，分工明确。

栈主要用来存方法的局部变量、方法参数和返回地址。每个线程有自己独立的栈，生命周期和线程一致。基本数据类型（如 int、char）和对象引用通常存在栈上，访问速度快，但空间小，一般就几 MB。

堆则是所有线程共享的一块区域，用来存放对象实例。比如你 new 出来的对象，都分配在堆上。堆的大小可以很大，通常是几百 MB 到几个 GB，由 JVM 参数控制。垃圾回收器主要管理的就是堆内存。

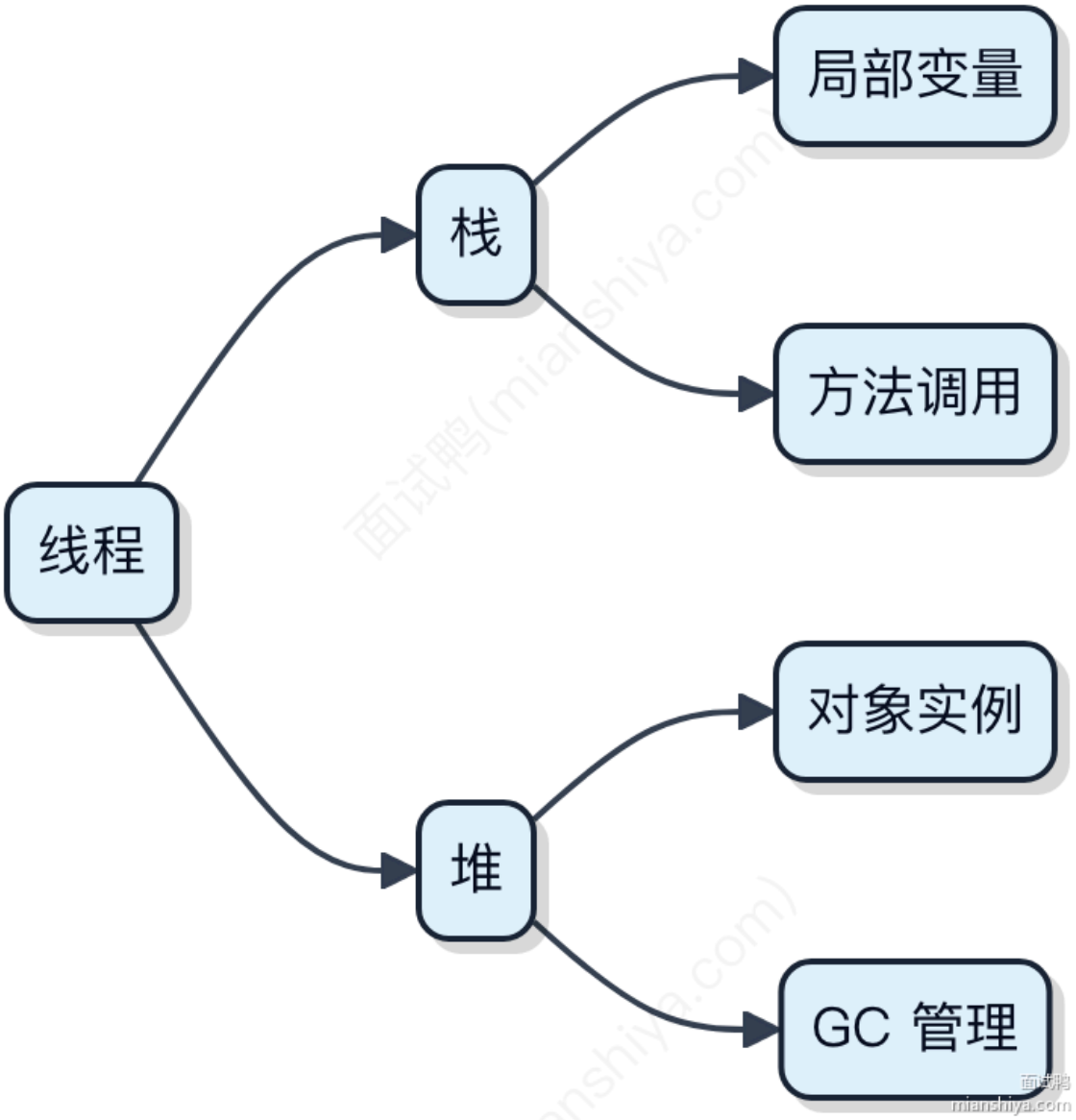
举个例子：

```
void method() {
    int a = 10;               // a 在栈上
    Object obj = new Object(); // obj 引用在栈上，new Object() 实例在堆上
}
```

方法执行完，栈帧弹出，a 和 obj 这两个变量就没了，但堆里的对象能不能回收，得看有没有其他引用指向它。

栈不会发生 GC，只有堆才会被垃圾回收。如果栈深度太深（比如递归没出口），会报 StackOverflowError；如果堆内存不足，则抛 OutOfMemoryError。

简单说，栈管“临时变量”，堆管“对象本身”。一个方法调用的过程，其实是栈帧不断压入弹出的过程，而对象创建全靠堆撑着。



为什么 Java 新生代被划分为 S0、S1 和 Eden 区？

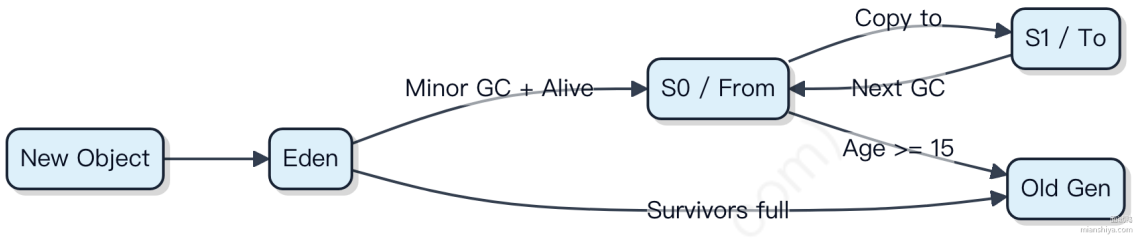
新生代这么划分，主要是为了高效处理对象的生命周期差异，用**复制算法**实现垃圾回收。

大多数对象朝生夕死，刚创建就变成垃圾。Eden 区就是专门用来分配新对象的地方。当 Eden 满了，触发 Minor GC，把还活着的对象复制到 S0 或 S1 中的一个。

S0 和 S1 互为 **From** 和 **To**，每次 GC 时，存活对象从一个 Survivor 区（From）复制到另一个（To），同时年龄加 1。下一次 GC 时角色翻转。这样设计的好处是能自动整理内存，避免碎片。

对象在 Survivor 区每经历一次 GC，年龄+1，达到阈值（默认15）就晋升到老年代。如果某次需要复制的对象太多，Survivor 放不下，也会提前晋升。

这种分法让 GC 只在小范围内运行，速度很快。像 CMS、G1 这些收集器底层也借鉴了这个思路。



要是没有 S0/S1 的双缓冲机制，光靠 Eden 和一个 Survivor，要么没法运作，要么效率低下。这两个区看着浪费空间，其实是为复制算法服务的关键设计。

什么是 Java 中的常量池？

Java 里的常量池主要管两件事：类加载时的字面量和符号引用，还有运行时生成的字符串复用。

类文件里有个常量池，存着编译期确定的字面量，比如 `"hello"` 这种字符串，还有类名、方法名这些符号信息。JVM 加载类的时候会把这部分搬到方法区，叫**运行时常量池**，属于元空间的一部分。

真正影响性能的是字符串常量池。它本质是个 `HashMap<String, String>`，专门存字符串对象的引用，确保内容相同的字符串只保留一份。比如你写 `"abc"`，JVM 先去池子里找有没有，有就直接返回，没有就放进去再返回。这个池子从 JDK7 开始挪到了堆内存，方便 GC 管理。

通过 `intern()` 方法可以手动把字符串对象加入池中。比如 `new String("abc").intern()`，如果池里没有 `"abc"`，就会把堆里这个对象的引用存进池子。

```
String a = "abc";                    // 直接从常量池拿
String b = new String("abc");        // 堆里新建对象
String c = b.intern();               // 把 b 对应的值入池，返回池引用
System.out.println(a == c);          // true，a 和 c 都指向池中同一个
```

常量池能省内存，减少重复字符串创建，但 intern 操作本身有哈希计算和同步开销，频繁调用可能拖慢性能。

什么是 Java 中的 JIT（Just-In-Time）？

JIT 其实就是 Java 虚拟机在程序运行时，把热点的字节码动态编译成机器码的过程。你写的 Java 代码先被 javac 编译成字节码，然后由 JVM 解释执行。但解释执行慢啊，所以 JIT 就来了。

它会监控哪些方法被频繁调用，也就是所谓的“热点代码”。一旦发现，就立马把这些字节码编译成当前 CPU 架构直接能跑的机器码，下次调用就直接执行本地代码，速度飞起。

1）触发条件一般是方法被调用一定次数，比如默认是 10000 次（Client VM），这个叫热点探测
2）编译后的机器码缓存起来，但可能因为继承、类加载等变化而失效，需要重新编译
3）不同 JVM 实现有差异，HotSpot 就用了分层编译：C1 编译快但优化弱，C2 优化强但耗时长，结合使用

```
// 你看这种循环调用的方法，很容易被 JIT 优化
public static long sum(int n) {
    long result = 0;
    for (int i = 0; i < n; i++) {
        result += i;
    }
```

```
    return result;
}
```

不用 JIT 的话，Java 性能压根没法跟 C++ 这种静态编译语言比。有了它，很多场景下性能差距大幅缩小，尤其是长期运行的服务，比如 Tomcat、Spring Boot 应用。

不过 JIT 也有代价，编译过程本身要消耗 CPU 和内存，启动阶段还感受不到，这就是为什么有些服务有“预热”过程。

什么是 Java 的 AOT（Ahead-Of-Time）？

AOT 指在程序运行前就把字节码编译成机器码，和传统的 JIT（运行时编译）反过来。Java 长期依赖 JVM 的即时编译，启动慢、内存占用高，尤其在云原生场景下成了短板。

GraalVM 提供了 **native-image** 工具实现 AOT 编译，能把 Java 程序打成一个独立的可执行文件。这个过程会做静态分析，把用到的类、方法、反射信息全提前编译进去。结果是启动速度从秒级压到几十毫秒，内存占用也直接砍掉一大半。

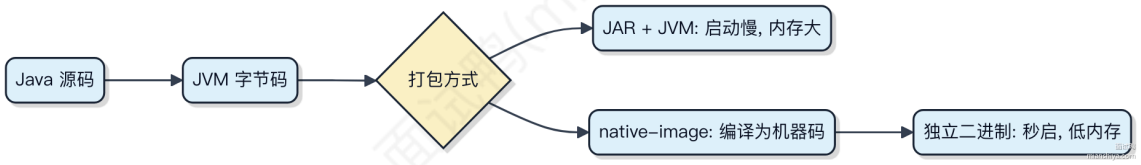
但代价不小。首先，所有动态特性得手动配置，比如反射、动态代理、资源加载，都得通过配置文件告诉编译器。其次，不支持 Class.forName 这种运行时加载类的操作，连 `java.lang.Thread` 的某些 API 都受限。Spring Boot 早期版本完全跑不起来，直到推出 Spring Native 才逐步适配。

适合用 AOT 的场景很明确：短生命周期、高并发拉起的服务，比如 Serverless 函数。FaaS 平台要求百毫秒内启动，传统 JVM 压根扛不住，这时候 native image 就能派上用场。但常规的长驻 Web 服务，JIT 的热点优化能力还是更强，AOT 反而可能跑不过。

代码层面没啥特别，加个插件就行：

```
// pom.xml 片段
<plugin>
    <groupId>org.graalvm.buildtools</groupId>
    <artifactId>native-maven-plugin</artifactId>
</plugin>
```

构建后生成的是二进制文件，直接 `./app` 启动，不依赖 JRE。



为什么 Java 的垃圾收集器将堆分为老年代和新生代？

Java 的堆内存分代设计不是凭空来的，是基于大量观测数据做出的工程妥协。你得先知道一个现象：绝大多数对象都是朝生夕死的，研究发现 98% 以上的对象在创建后很快就会变成垃圾。

1）新生代专门放这些短命对象。它又分成 Eden 区和两个 Survivor 区（S0/S1）。大多数对象直接在 Eden 分配，GC 时活下来的复制到 S 区，熬过几次 GC 还活着的，就升级去老年代。这种复制算法在存活对象少的时候效率很高。

2）老年代的对象都比较“长寿”，可能是一些缓存、静态变量引用的对象。这里空间大，用标记-整理或标记-清除这类适合高存活率场景的算法更合适。

不分代的话，每次扫描整个堆，GC 停顿时间根本扛不住。分代之后，Minor GC 只扫新生代，频率高但速度快，一般几十到几百毫秒。Full GC 才扫老年代，虽然慢但触发少。

像 CMS、G1 其实都在尝试打破严格分代，但新生代+老年代这个基本模型到现在还是主流。ZGC 和 Shenandoah 想搞全堆并发回收，可代价也不小。

Java 中的强引用、软引用、弱引用和虚引用分别是什么？

Java 的引用类型不止一种，除了我们天天用的普通引用，还有几种特殊引用，它们的区别主要体现在 **垃圾回收器** 对它们的处理策略上。

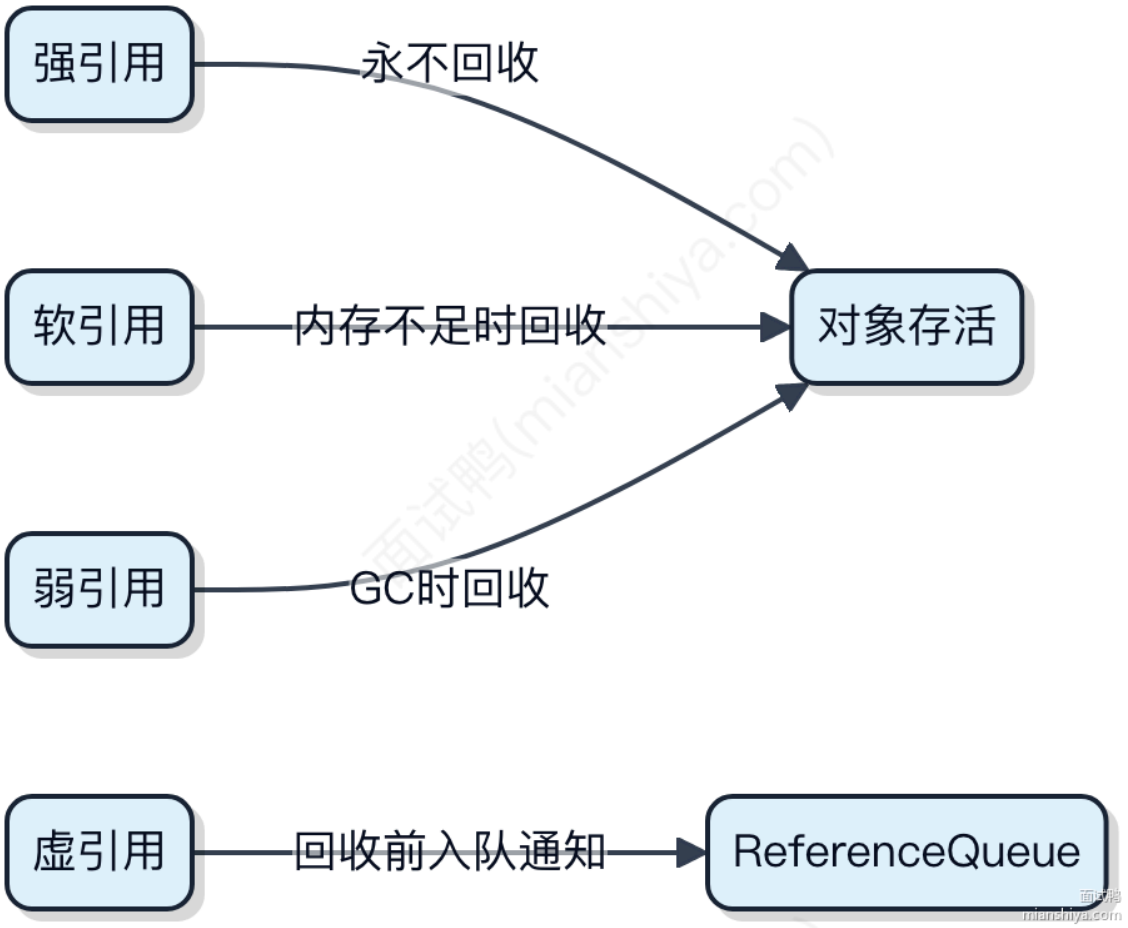
1）强引用就是平时最常见的，比如 `Object obj = new Object()` ，只要这行代码还挂着，哪怕内存快满了，对象也不会被回收。这是最硬的关联。

2）软引用在内存充足时不会被清理，一旦系统快撑不住了，JVM 才会把它标记的对象干掉。适合做缓存，比如你用 `SoftReference<byte[]>` 来缓一个大文件内容，内存不够自动释放，够用就留着。Guava Cache 内部就用到了这个特性。

3）弱引用比软引用更短命，只要发生 GC，不管内存够不够，直接回收。典型场景是 `ThreadLocal` 里的 `Entry` ，用的是 `WeakReference<ThreadLocal>` ，防止内存泄漏。但注意，只是 key 被弱引用，value 不是，所以还得手动清理。

4）虚引用最特别，它几乎不控制对象生命周期，唯一作用就是能在这个对象被回收时收到一个系统通知。必须配合 `ReferenceQueue` 使用，一般用于追踪 GC 行为，像 NIO 中的直接内存管理就靠它触发清理。

面试鸭(mianshiya.com)



记住，除了强引用，其他都得通过 `get()` 拿对象，而且随时可能返回 null，用的时候得判空。

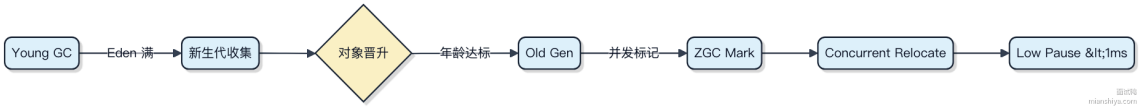
[Java 中常见的垃圾收集器有哪些？]

Java 的垃圾收集器得按年代来看，不同年代用的策略和实现不一样。早期 CMS 主要解决老年代回收停顿时间长的问题，用标记-清除算法，并发收集，但容易产生碎片，搞不定的时候还得靠 Serial Old 收尾。

G1 收集器是 JDK 9 默认的，主打“低延迟”，把堆拆成多个 Region，优先回收垃圾最多的区域，用写屏障维护 Remembered Set 来追踪跨区引用，避免全局扫描。它在大部分场景下能控制 STW 时间在目标范围内，适合大堆、多核机器。

到了 JDK 11+，ZGC 和 Shenandoah 是新主力。它们都追求亚毫秒级停顿，支持几十 GB 甚至 TB 级堆。ZGC 用着色指针 + 读屏障，在标记和转移阶段几乎不暂停应用线程，停顿基本压在 1ms 以内。Shenandoah 类似，但它不依赖着色指针，而是额外加了个转发指针。

1）新生代这边，Parallel Scavenge 注重吞吐量，适合后台批处理服务
2）CMS 已经被废弃，ZGC 在生产环境越来越常见，尤其对延迟敏感的系统比如交易、实时计算



选型上，一般服务用 G1 足够了，要是堆特别大、要求极低延迟，直接上 ZGC。不过 ZGC 对操作系统有要求，比如 Linux 上需要透明大页关闭。

Java 的 CMS 垃圾回收流程是怎样的？

CMS（Concurrent Mark Sweep）是 Java 老年代的一种垃圾回收器，目标是尽可能减少停顿时间，适合对延迟敏感的应用，比如 Web 服务中的 Tomcat 或 Dubbo 服务节点。

它整个流程分七个阶段，但只有部分阶段需要 STW（Stop-The-World）。

1）初始标记：会暂停所有应用线程，只标记从 GC Roots 直接可达的老年代对象。这个阶段非常快，因为只处理根节点的直接引用。

2）并发标记：和用户线程同时运行，遍历并标记所有可达的对象。这一步耗时较长，但不阻塞业务代码。

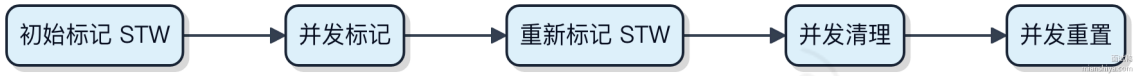
3）重新标记：再次暂停应用线程，修正并发期间因对象变化导致标记不准确的问题。主要是处理并发标记阶段中对象引用变动的情况，比如老年代对象被年轻代对象引用的场景。

4）并发清理：和用户线程一起运行，回收未被标记的垃圾对象。清理完成后，会产生内存碎片。

5）并发重置：为下一次 CMS 回收集准备数据结构，也是并发执行。

CMS 不会压缩内存，所以用久了会出现**内存碎片**。当无法找到连续空间分配大对象时，即使总空闲空间足够，也会触发 Full GC。这个问题在高负载长时间运行的服务中特别明显，比如订单系统跑几天后突然 Full GC 频繁。

另外，CMS 依赖更多的 CPU 资源，因为并发阶段要和应用争抢 CPU 时间。如果服务器本身负载就高，反而可能拖慢整体性能。



现在主流 JDK 已经弃用 CMS，推荐使用 G1 或 ZGC 来替代。

什么是 Java 中的直接内存（堆外内存）？

Java 里的直接内存，也叫堆外内存，是 JVM 堆以外的内存空间，由操作系统直接管理。它不归 GC 管，创建和销毁得靠开发者自己手动控制，典型的就是通过 `ByteBuffer.allocateDirect()` 来申请。

1）这块内存最大的好处是能减少数据在 JVM 堆和 native 堆之间拷贝。比如网络传输或文件读写时，使用直接内存可以让底层 OS 直接操作这块区域，避免多一次复制，提升 I/O 性能。Netty 就大量用了直接内存来处理网络数据包。

2）但代价也很明显。分配和回收成本高，毕竟要跨 JNI 调用系统 API。而且一旦泄漏，GC 发现不了，只能靠监控和手动释放，容易出问题。-XX:MaxDirectMemorySize 可以限制最大用量，超了会抛 OutOfMemoryError。

3）典型的使用场景包括高性能通信框架（如 Netty）、大文件 MappedByteBuffer 映射、或者需要长期存活且频繁交互 native 代码的场景。

简单说，直接内存就是拿编程复杂度换性能，适合对延迟敏感、I/O 密集的场景，但别乱用，搞不定反而拖垮稳定性。

你了解 Java 的类加载器吗？

Java 的类加载器不是简单的文件读取工具，它负责在运行时动态将 .class 文件加载到 JVM 中，并转换成 `java.lang.Class` 对象。

整个机制基于**双亲委派模型**。当一个类加载请求到来时，当前类加载器不会自己先去加载，而是交给它的父类加载器处理。只有当父类加载器无法完成加载（比如在指定路径找不到类），子加载器才会尝试自己加载。这种设计保证了像 `java.lang.Object` 这种核心类不会被自定义加载器篡改，保障了安全性和一致性。

JVM 内置三类加载器： 1）启动类加载器（Bootstrap ClassLoader）由 C++ 实现，加载 `$JAVA_HOME/lib` 下的核心类库，比如 `rt.jar` 。 2）扩展类加载器（Extension ClassLoader）加载 `$JAVA_HOME/lib/ext` 目录下的类。 3）应用程序类加载器（Application ClassLoader）加载用户 classpath 指定的类。

你可以自定义类加载器，继承 `ClassLoader` 并重写 `findClass()` 方法。典型场景包括热部署（如 Tomcat 隔离各个 Web 应用）、模块化加载（OSGi）或远程类加载。

```
public class CustomClassLoader extends ClassLoader {
    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        byte[] classData = loadClassData(name);
        if (classData == null) throw new ClassNotFoundException();
        return defineClass(name, classData, 0, classData.length);
    }
}
```

打破双亲委派的少数情况是线程上下文类加载器，用来解决父类加载器需要加载应用级别的实现，比如 JNDI 服务加载数据库驱动。

你了解 Java 的逃逸分析吗？

逃逸分析是 JVM 在运行时的一种优化手段，它用来分析对象的动态作用域。如果发现一个对象只在某个方法内部被使用，没有“逃逸”到其他线程或方法，JVM 就可以做些激进的优化。

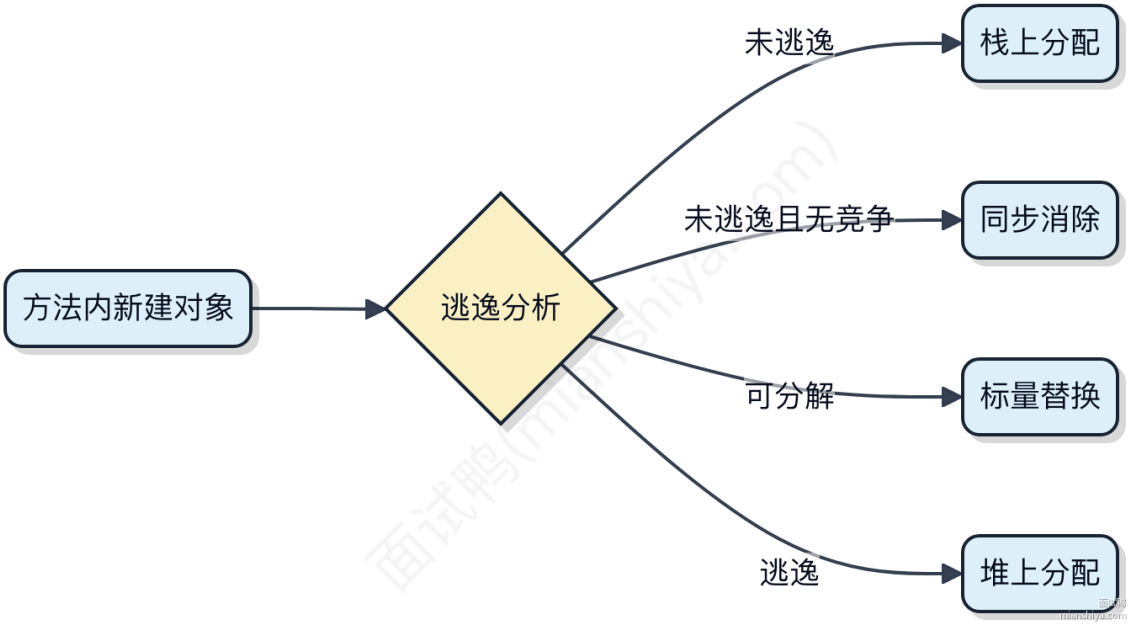
这种情况下，对象其实没必要非得分配在堆上。**栈上分配**就成了可能，由调用栈负责生命周期，方法结束直接回收，省去了 GC 的开销。像一些短生命周期的对象，比如 StringBuilder、临时包装类，经常能被优化掉。

还有一个常见优化是**同步消除**。如果分析出加锁的对象只被单线程访问，synchronized 块就会被直接去掉。比如方法里新建的对象作为锁，外面根本拿不到，那这把锁压根不经过竞争，JVM 会把它干掉。

再就是**标量替换**。对象本来要分配空间存字段，但如果拆成一个个基本类型（标量），并且这些值可以放在 CPU 寄存器里，那连栈上分配都省了，性能更高。

不过这些都依赖 JIT 编译器的实现程度，HotSpot 对逃逸分析的支持在不断改进，但不是所有场景都能触发。你写代码的时候也不用刻意绕着它来，JVM 自己会判断。

要不要开启看实际效果，-server 模式默认打开，-XX:+DoEscapeAnalysis 控制开关。重点是理解它让“对象一定在堆上”这个认知不再绝对。



Java 中如何判断对象是否是垃圾？不同实现方式有何区别？

判断对象是否可回收，关键看它是不是还被程序需要。JVM 主要靠两种方式：引用计数和可达性分析。

引用计数很简单，每个对象有个计数器，有地方引用它就加一，引用失效就减一。归零说明没人用，就是垃圾。Python 用这个，但 Java 的主流虚拟机压根不经过这路子，因为解决不了循环引用的问题。比如 A 引 B，B 又引 A，没外部引用了本该回收，但计数都不为零，搞不定。

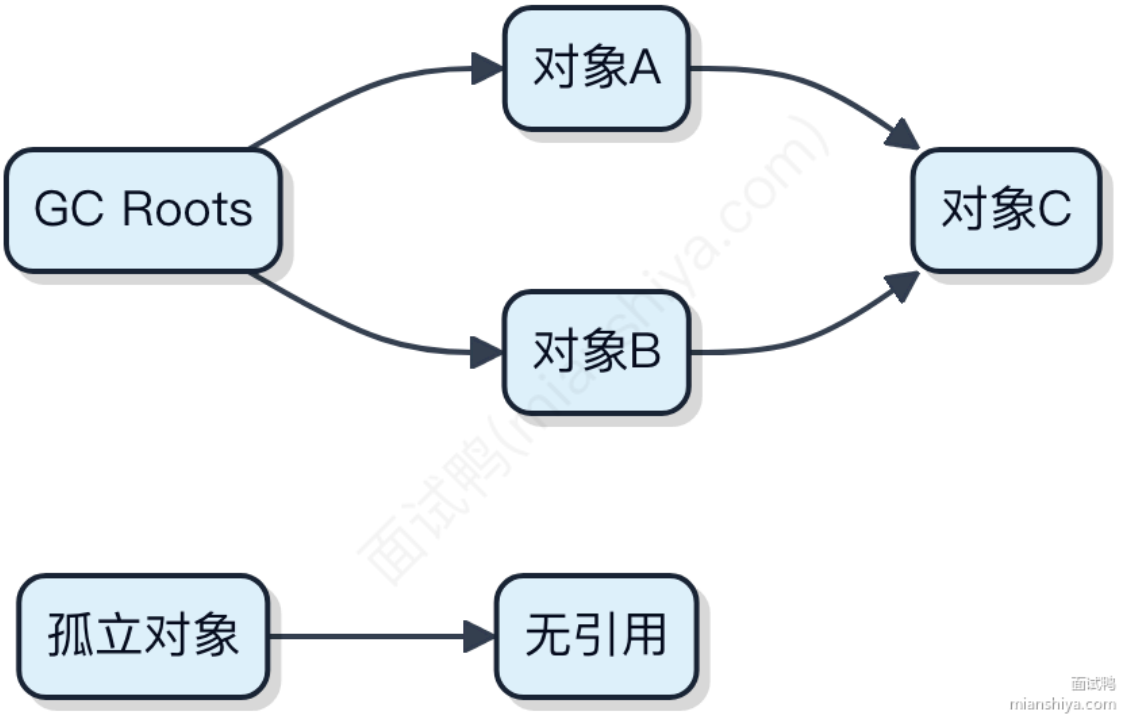
Java 用的是 **可达性分析**。从 GC Roots 开始往下搜，能被遍历到的对象都算活着，搜不到的才是垃圾。GC Roots 包括线程栈里的本地变量、静态变量、常量这些起点。这种方式能处理复杂引用关系，是现在 JVM 的标准做法。

代码层面其实不直接暴露这些，但你写的时候得心里有数。比如静态集合一直 hold 着对象，就可能导致内存泄露，因为它成了 GC Roots 一部分，对象一直可达。

```
static List<Object> cache = new ArrayList<>();
// 如果不断往里 add，不清理，这些对象永远可达，哪怕你再也不用了
```

1）引用计数实现简单，开销小，但有循环引用硬伤
2）可达性分析能准确识别不可达对象，配合各种收集器高效回收，是 HotSpot 的选择

要不要用哪种，其实在虚拟机层面早就定好了。我们做应用层开发，更重要的是避免人为制造“假活”对象，比如过度使用静态缓存、监听器没注销这些。



图里 E 对象没有路径从 GC Roots 到达，会被判定为可回收。

你了解 Java 的 ZGC（Z Garbage Collector）吗？

ZGC 是 JDK 11 引入的低延迟垃圾收集器，目标是让停顿时间**压根不经过** 10ms，哪怕堆内存扩到 16TB 也能扛住。它适合对延迟敏感的场景，比如用 Netty 做网关、Flink 实时计算这类系统。

ZGC 的核心是**读屏障**和染色指针。它把 GC 信息直接编码进指针里，用地址的几位标记对象是否被重定位、是否可达。这样线程访问对象时，通过读屏障自动修正指针，不用整个 STW 扫堆。

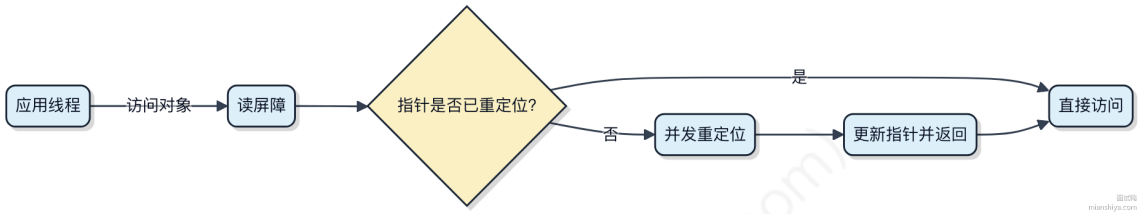
它整个回收过程几乎全并发，包括标记、转移、重定位。只有初始标记和最终选择性重定位会短暂 STW，但时间固定，和堆大小无关。

ZGC 不分代，所有对象一视同仁。这在小对象多、生命周期短的场景可能不如 G1 高效，毕竟 G1 可以优先回收年轻代。所以如果你的应用是典型 Web 服务，95% 对象活不过一次 Young GC，那还是 G1 更合适。ZGC 更适合大堆、长生命周期对象多的场景，比如缓存系统或者超大状态的流处理。

代码上不需要特别配置，启动参数加上：

```
-XX:+UseZGC -Xmx16g
```

就能跑起来，JVM 会自动管理。



简单说，ZGC 是靠**读屏障**+染色指针实现超低延迟，堆越大优势越明显，但小对象场景可能反而拖累吞吐。

Java 中的 young GC、old GC、full GC 和 mixed GC 的区别是什么？

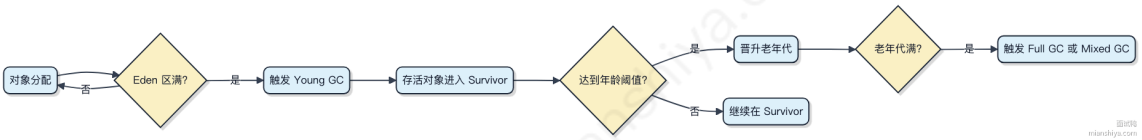
年轻代 GC（Young GC）发生在堆的年轻代区域，对象优先在 Eden 区分配，当 Eden 区满时触发。存活对象会被复制到 Survivor 区，经历过多次 Young GC 仍存活的对象会晋升到老年代。这个过程基本是复制算法，速度快，频率高，一般几十到几百毫秒一次。

老年代 GC（Old GC）特指只清理老年代的垃圾回收，但 CMS 等少数收集器才会单独触发。多数情况下老年代不会单独回收，而是伴随 Full GC 一起进行。

Full GC 是整个堆和方法区的全面清理，包括年轻代、老年代、元空间（或永久代）。它会暂停所有应用线程（Stop-The-World），耗时较长，可能持续几百毫秒甚至几秒。频繁 Full GC 通常意味着内存泄漏或堆配置不合理。

Mixed GC 是 G1 收集器特有的机制，可以同时回收年轻代和部分老年代 Region。它基于预测模型选择垃圾最多的 Region 优先回收，避免全堆扫描，控制停顿时间在预期范围内，比如 -XX:MaxGCPauseTimeMillis=200 设置目标为 200ms。

1）Young GC：仅年轻代，高频低开销
2）Old GC：仅老年代，少见，CMS 可独立触发
3）Full GC：全堆 + 元空间，STW 时间长
4）Mixed GC：G1 特有，年轻代 + 部分老年代 Region



Java 的 G1 垃圾回收流程是怎样的？

G1 回收不是简单的一次性动作，它把堆切成多个 Region，回收时优先挑垃圾多的区域下手，整体走的是“增量整理”路线。

初始标记这一步很快，就标记一下 GC Roots 直接能碰到的对象，STW 时间极短，一般也就几毫秒。接下来是并发标记，G1 会顺着这些根对象一路扫描下去，这个阶段和应用线程一起跑，不暂停。

当大部分标记做完后，再来一次最终标记，处理那些并发期间变动的对象引用，这时候才真正暂停。最后是清理阶段，G1 统计每个 Region 的垃圾占比，为后续的混合回收做准备。

真正执行回收是在 Young GC 和 Mixed GC 阶段。Young GC 只清理年轻代的 Region，而 Mixed GC 会把年轻代和部分老年代的 Region 一起回收。它用复制算法，把存活对象搬到别的空 Region 里，顺便压缩内存，避免碎片。

代码上你不用手动触发，JVM 自己调度。但可以通过参数控制行为：

```
-XX:+UseG1GC -XX:MaxGCPauseTimeMillis=200 -XX:G1HeapRegionSize=16m
```

重点是 **Remembered Set**，每个 Region 都有个 RSet 记录外部对象对它的引用，这样回收时不用扫描整个堆，压根不经过非目标 Region，效率高得多。

Java 中的 CMS 和 G1 垃圾收集器如何维持并发的正确性？

并发收集器要解决的根本问题是：用户线程在跑，GC 线程也在跑，对象引用关系乱变，怎么保证标记不漏不重？

CMS 和 G1 都靠“读写屏障 + 增量更新/原始快照”这套组合拳来维持正确性，但具体打法不一样。

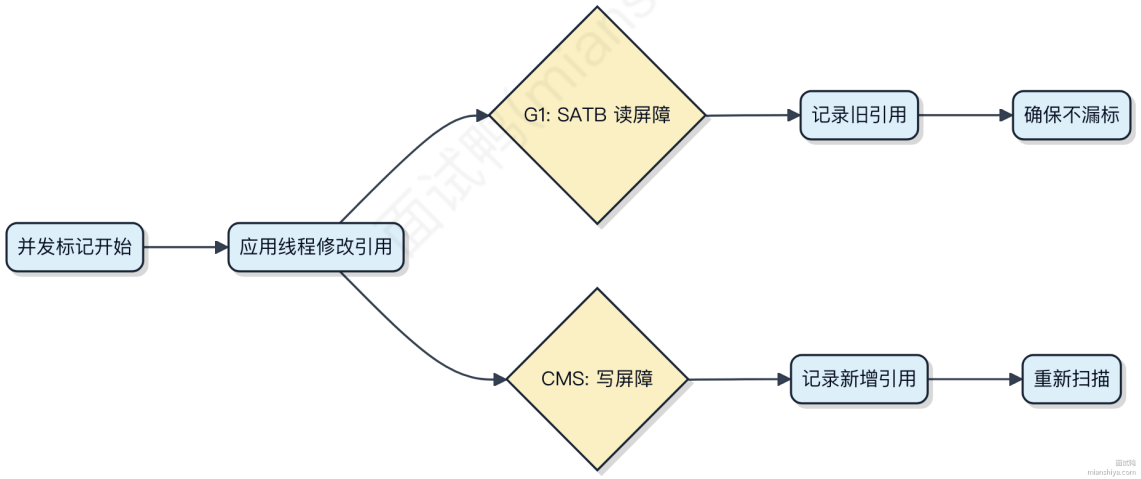
CMS 用的是**增量更新（Incremental Update）**。它关注的是：已经被标记过的对象，会不会重新被一个未被标记的对象引用？如果会，那就可能漏标。CMS 在写操作时加写屏障，只要发现有引用关系从 null 变成指向老对象，就记录下来，后续重新扫描这些变动点。CMS 的读屏障基本不用。

G1 走的是另一条路，用的是**原始快照（SATB, Snapshot-At-The-Beginning）**。它在并发标记开始时拍个“快照”，认为那一刻活着的对象一定得活。之后如果某个引用被修改，G1 会在旧引用断开前用读屏障把原值记下来，确保即使新对象没被标记，也不会漏掉这条路径。这种机制让 G1 的标记阶段更高效，漏标风险更低。

代码上你基本看不到这些，是 JVM 自动插的内存屏障指令，比如：

```
// 伪代码，实际由 JVM 在 putfield 等字节码插入
if (g1BarrierEnabled && oldValue != null) {
    logWriteBefore(oldValue); // SATB 记录旧值
}
```

总结一句：CMS 靠写屏障+增量更新防漏，G1 靠读写屏障+SATB 快照兜底，都是为了在并发场景下搞一次安全又高效的标记。



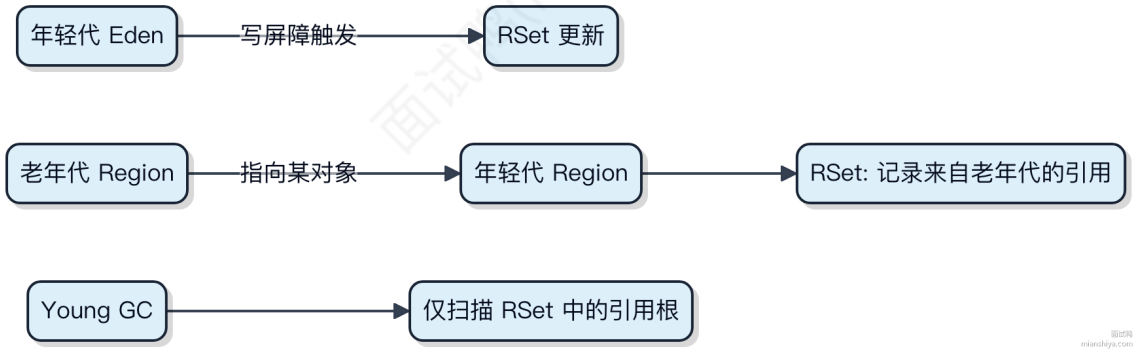
为什么 G1 垃圾收集器不维护年轻代到老年代的记忆集？

G1 垃圾收集器其实不是不维护记忆集，而是换了一种更高效的方式处理跨代引用。年轻代和老年代之间的引用问题，在 G1 里是靠 **记忆集（Remembered Set, 简称 RSets）** 来解决的，但它只在**目标区域（比如老年代的某个 Region）上维护**，记录的是谁引用了我，而不是从源头去维护。

这就意味着，当年轻代的 Region 被回收时，不需要扫描整个老年代来找引用，只需要看相关 Region 的 RSet 中有没有来自老年代的卡片标记。这种设计把原本 O(整个老年代) 的扫描成本，降到了 O(活跃的引用 Region 数量)，效果非常明显。

每个 Region 配一个 RSet，存储其他 Region 指向它的引用卡片。写屏障在对象引用更新时，会异步把信息记录到对应的目标 RSet 中。这样 GC 时，比如 Young GC，只需要根据 RSet 快速找出跨代引用根，压根不经过全堆扫描。

代码层面没有直接暴露 RSet 操作，它是 JVM 内部自动管理的结构。但你可以通过参数 `-XX:+PrintGCDetails` 观察 GC 日志里的 `RSets` 扫描耗时，来判断其开销。



RSet 的空间换时间策略，让 G1 在大堆场景下能扛住频繁的跨代引用扫描，脏活累活交给写屏障和并发线程慢慢做。

Java 的 CMS 垃圾回收器和 G1 垃圾回收器在记忆集的维护上有什么不同？

CMS 和 G1 虽然都用**记忆集**（Remembered Set）来记录跨区域的引用，避免全局扫描，但它们的实现方式和维护策略差别挺大。

CMS 的记忆集是“卡表”（Card Table），把堆分成 512 字节的卡页。当某个对象引用了另一个区域的对象时，对应的卡页被标记为“脏”。写操作通过写屏障触发，标记卡页。但 CMS 不会立刻清理这些脏页，而是等到并发标记阶段再去处理。这导致它可能需要扫描一些实际上没有有效跨代引用的脏卡页，效率不够高。

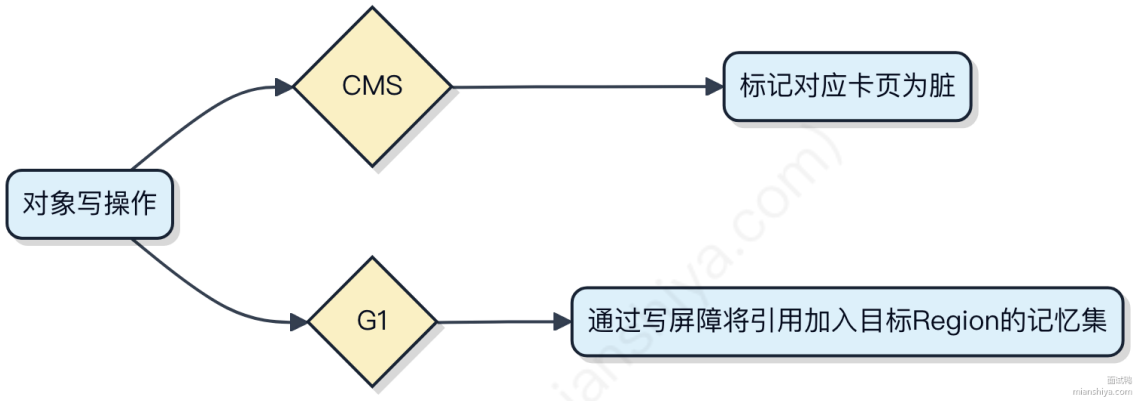
G1 的记忆集是基于哈希表的结构，每个 Region 都有自己的 Remembered Set，存的是指向它的其他 Region 中的对象引用。G1 写屏障在发生引用变更时，直接把引用信息以 `<Region, offset>` 的形式投递到目标 Region 的记忆集中。这样每个 Region 的记忆集只记录精准的外部引用，回收时不用扫描整个卡表。

关键区别在于：

1）CMS 的卡表是粗粒度的，一个卡页可能包含多个对象，只要有一个跨代引用就被标脏，存在“伪共享”。

2）G1 的记忆集是细粒度的，记录的是具体哪个 Region 的哪个位置引用了自己，回收时精确打击。

3）G1 在写屏障中做的工作更多，属于“增量式更新”（Incremental Update），靠并发线程异步处理，把脏活累活拆开干，避免停顿。



所以 G1 的记忆集维护成本更高，但回收阶段更高效，适合大堆；CMS 维护简单，但回收时要扫更多无效区域，对堆大小敏感。

为什么 Java 8 移除了永久代（PermGen）并引入了元空间（Metaspace）？

永久代在 Java 7 及之前负责存储类的元数据，比如类名、方法、字段信息等。它有个致命问题：大小固定，默认最大就 80 多 MB，加载大量类时很容易撑满，导致 **OutOfMemoryError: PermGen space**。尤其在运行时生成类的场景，像使用 CGLIB 做动态代理的 Spring AOP，或者部署多个应用的 Tomcat 环境，这个问题特别常见。

元空间就是为了解决这个痛点来的。最关键是它**不再使用 JVM 堆内存**，而是直接用操作系统的本地内存（native memory）。这意味着只要系统还有内存，元空间就能扩展，基本不会因为类多就 OOM。默认情况下，元空间会一直扩容，直到系统内存耗尽。

另一个好处是 GC 配合更顺了。以前永久代里的类信息和其他堆对象混在一起，GC 清理时得全堆扫描，效率低。元空间独立出来后，类的元数据由专门的机制管理，配合 **Full GC 才触发回收**，减少了干扰。

当然，元空间也不是无限的。可以通过 `-XX:MaxMetaspaceSize` 限制上限，否则可能把系统内存耗光。如果不设，它会自动调整。

简单说，从永久代到元空间，本质是把类的元数据从受限的 JVM 内存移到了更灵活的本地内存，解决了容量瓶颈和 GC 效率问题，更适合现代动态生成类的应用场景。

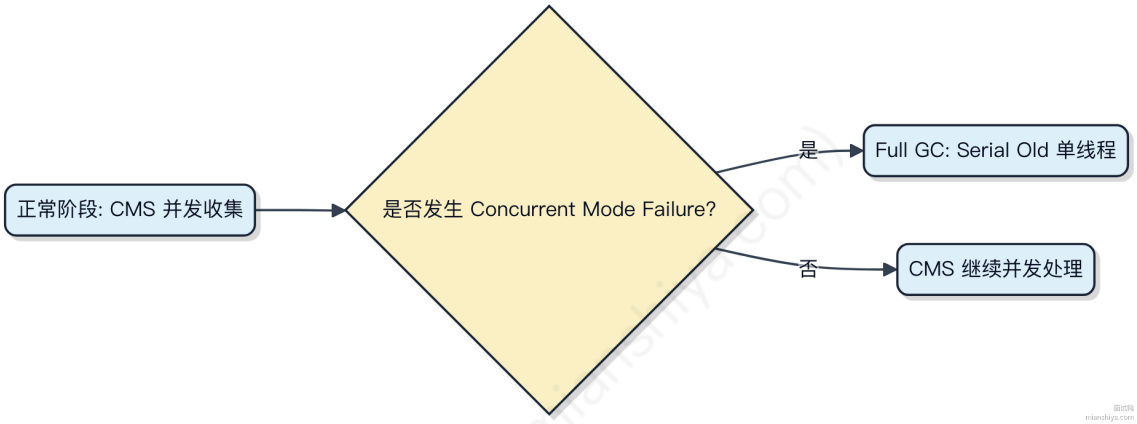
为什么 Java 中 CMS 垃圾收集器在发生 Concurrent Mode Failure 时的 Full GC 是单线程的？

CMS 收集器设计目标是降低停顿时间，大部分工作和用户线程并发执行。但一旦触发 Concurrent Mode Failure，意味着在 CMS 并发标记或清理过程中，老年代空间不足以容纳新晋升的对象。

这时候 JVM 会退回到 Serial Old 收集器来执行 Full GC。Serial Old 是单线程的，原因很实际：
1）它是在紧急情况下被启用的备用方案，并发失败说明系统已经处于内存严重不足的状态，资源紧张。
2）多线程 GC 需要额外的协调开销，比如并行线程调度、数据结构同步等，在这种危机场景下反而可能让情况更糟。
3）实现复杂度低，代码稳定可靠，适合兜底使用。

你可以把 CMS 看作是“日常保洁”，一边运行程序一边打扫堆内存。但当垃圾太多来不及清理时，系统必须彻底暂停，进入“大扫除模式”。这个模式不需要花哨，并且要尽量减少额外负担，所以用最简单稳定的单线程方式完成。

虽然从 JDK 7 开始可以通过配置 `-XX:+UseParallelOldGC` 启用并行 Full GC，但 CMS 本身发生 Concurrent Mode Failure 时默认仍走 Serial Old 路径，除非明确替换为 Parallel Old。



避免这个问题的关键是合理设置 `-XX:CMSInitiatingOccupancyFraction` ，提前触发 CMS，留足空间。

JVM 垃圾回收时产生的 concurrent mode failure 的原因是什么？

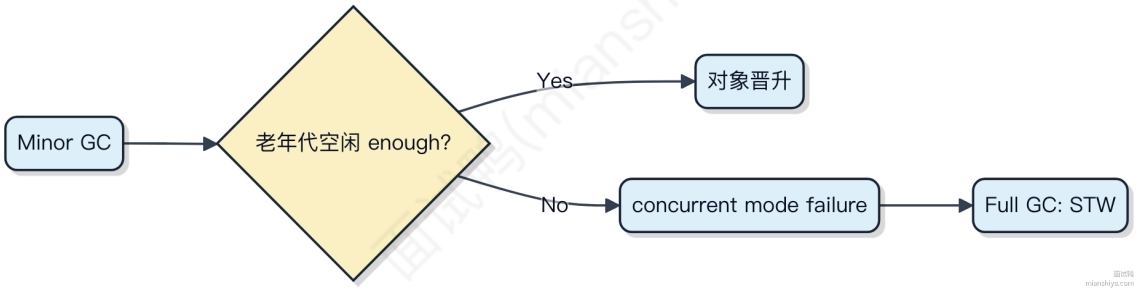
并发回收期间发生 concurrent mode failure，根本原因是老年代没足够的空间容纳从年轻代晋升上来的对象，导致回收线程搞不定，必须暂停所有应用线程做 Full GC。

CMS 和 G1 这类并发收集器，都是让 GC 线程和用户线程一起跑，减少停顿。但它们有个前提：老年代得留够空间，用来放并发期间新晋升的对象。如果回收过程中发现老年代剩余空间压根不经过分配担保就快满了，就会触发 concurrent mode failure。

常见原因有两个：

1）老年代碎片太多，虽然总空闲空间够，但没法分配连续大对象，CMS 又没开启压缩，只能失败。 2）年轻代一次 Minor GC 晋升的对象太多，老年代瞬时扛不住，比如有大对象直接进入老年代，或者 Survivor 区太小待不住。

避免这个问题，一般要控制晋升节奏。比如增大 Survivor 区、提前触发老年代回收（调低 `CMSInitiatingOccupancyFraction` ），或者直接换 G1，它会优先回收垃圾最多的 Region。



简单说，并发回收的底气是“我慢慢收”，但一旦老年代紧急缺人，就得叫停一切，来一次彻底清扫。

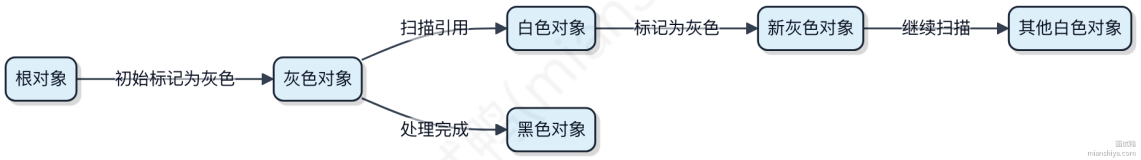
什么是三色标记算法？

三色标记算法是 JVM 垃圾回收中用于找出存活对象的核心手段，它用三种颜色来表示对象的标记状态。整个过程不用停机，能和用户线程并发跑。

1）白色：还没被垃圾回收器访问到的对象，认为是可回收的。 2）灰色：对象本身被标记了，但它的引用字段还没被扫描完，属于“正在处理中”。 3）黑色：对象已标记，且所有引用字段也都处理完毕，确定是存活的。

GC 开始时，所有对象都是白色。根对象（如栈变量、静态变量引用的对象）先被标记为灰色，放进待处理队列。然后从队列里取灰色对象，扫描它引用的字段，把引用的对象从白变灰，并把自己变成黑色。反复这个过程，直到灰色队列为空。最后剩下的白色对象就是不可达的垃圾，可以被回收。

并发标记有个大问题：用户线程在改引用关系，比如把一个已标记的黑色对象指向了一个白色对象，这会导致白色对象被误删。解决办法是写屏障，当发生这种赋值时，触发一段逻辑，把那个黑色对象重新变灰，让它再被扫描一次。CMS 和 G1 都用了类似机制，ZGC 则用的是另一种思路。



写屏障就像个监控探针，只要发现有引用更新，就按规则调整标记状态，确保标记正确性。这套机制让 GC 能在不停顿的情况下完成堆遍历。

什么情况下会触发 Java 的 Full GC？

老年代空间不足是最常见的 Full GC 触发原因。当年轻代晋升的对象体积太大，或者长期存活的对象逐渐填满老年代，回收后还是放不下，CMS 或 G1 就得回退到 Serial Old 这种单线程收集器来彻底清理，这时候整个应用就停了，也就是我们常说的 stop-the-world。

永久代或元空间满了也会触发。比如动态生成类的场景，像 CGLIB 做 AOP 时不断创建新类，元空间又没设上限，最后就会因为 Metaspace 空间不足引发 Full GC。这种情况在 Spring 动态代理用得多的服务里其实不少见。

主动调用 `System.gc()` 也可能触发，虽然 JVM 不一定立刻执行，但加上 `-XX:+DisableExplicitGC` 能禁掉这个行为，很多中间件比如 Kafka、RocketMQ 启动时都会加这个参数避免被外部搞乱节奏。

还有就是 CMS 的并发模式失败或晋升失败。并发清理期间如果老年代增长太快，来不及清，就会降级为 Full GC。同样，年轻代回收时如果 Survivor 区扛不住，大量对象提前进入老年代，也容易把老年代撑爆。

所以线上服务一般要监控老年代和元空间的使用趋势，配合 G1 或 ZGC 这类低停顿的收集器，才能抗住流量高峰。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*